CAFFEINE ENGINE

INTERNAL TECHNICAL REFERENCE

Core Systems, Mathematics, and Architectural Foundations

A formal specification covering the mathematical basis and implementation rationale of every subsystem in the Caffeine Engine. This document is the authoritative reference for contributors and serves as the planning specification for all future core and UI implementations.

Repository	`devscafecommunity/caffeine`
Standard	C++20, SDL3, ImGui
Platform	Windows / Linux / macOS (x64)
Revision	2026

This document is internal. It does not duplicate the public README or user guide; its sole purpose is deep technical understanding.

Abstract

The Caffeine Engine is a custom C++20 game engine built over SDL3, designed with explicit control over hardware, memory, and concurrency as primary objectives. This document provides the complete internal specification of its core systems: the primitive type layer, mathematical library (vectors, matrices, quaternions), custom memory allocators, generic container implementations, the Entity-Component-System (ECS) architecture, a work-stealing job system, a 2D rigid-body physics engine, a spatial audio system, a binary asset pipeline, and the software-rasterised 3D/Isometric scene viewport with its projection mathematics and transform gizmo framework.

Each system is described at the level required to understand its design decisions, prove its correctness, and extend it safely. Formulas are presented in their complete mathematical form; implementation details follow directly from the theory. This document is both a teaching resource and a binding specification: any proposed change to a described system must be reconciled with the invariants stated here before implementation begins.

Keywords: game engine, ECS, memory allocators, quaternions, work-stealing scheduler, impulse-based physics, software projection, asset pipeline, C++20.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Philosophy

Caffeine Engine is built under a single principle: *transparency*. Every byte that passes through the CPU must be accountable to the programmer. This philosophy manifests in three concrete decisions.

- **Zero-dependency standard library.** The engine replaces `std` containers with its own implementations tuned for game-loop access patterns (cache-friendly sequential iteration, allocator-aware growth, no hidden heap traffic).
- **Data-oriented design.** Entities are not objects; they are integer identifiers. Components are plain-old-data structs stored in contiguous typed pools. This layout maximises SIMD and cache line utilisation during system updates.
- **Explicit concurrency.** No implicit background threads. The job system is the single scheduling primitive; callers declare dependencies explicitly through barriers and handles.

1.2 Document Structure

Each subsequent chapter covers one major subsystem. The ordering follows the dependency graph: fundamental types and mathematics are presented first, followed by memory, containers, ECS, threading, domain systems (physics, audio), the asset pipeline, and finally the editor viewport. Each chapter concludes with a table of invariants that any future implementation must preserve.

1.3 Notation Conventions

Throughout this document, the following notation is used consistently.

- Scalars: italic Latin letters — a, b, t, θ.
- Vectors: bold lower-case — $\mathbf{v}, \mathbf{u}, \mathbf{n}$.
- Matrices: bold upper-case — $\mathbf{M}, \mathbf{R}$.
- Quaternions: calligraphic — $\mathcal{Q}$.
- Unit vectors: hat notation — $\hat{\mathbf{v}}$.
- Column-major storage is assumed unless stated otherwise.
- Code identifiers appear in `monospace`.

Chapter 2

Type System and Platform Abstractions

2.1 Primitive Types (`Types.hpp`)

All numeric types in the engine are defined as explicit-width aliases in `src/core/Types.hpp`. The rationale is binary portability: the C++ standard permits `int` to be 16- or 32-bit depending on the platform; the engine must never depend on implicit widths.

Table 2.1: Caffeine primitive type aliases

Alias	Underlying type	Width
`i8 / u8`	`int8_t / uint8_t`	8 bit
`i16 / u16`	`int16_t / uint16_t`	16 bit
`i32 / u32`	`int32_t / uint32_t`	32 bit
`i64 / u64`	`int64_t / uint64_t`	64 bit
`f32`	`float`	32 bit IEEE 754
`f64`	`double`	64 bit IEEE 754
`usize`	`std::size_t`	Platform pointer-width
`isize`	`std::ptrdiff_t`	Platform pointer-width

Every width is validated by `static_assert` at compile time. The consequence is that *inclusion of `Types.hpp` is the first gate for all cross-platform build failures*: if the platform violates any width, the translation unit refuses to compile.

2.2 Compiler Abstraction (`Compiler.hpp`)

Platform-specific compiler directives are isolated in a single header so that all engine source files are compiler-agnostic. The macros provided include `CF_INLINE` (`__forceinline` on MSVC; `__attribute__((always_inline))` on GCC/Clang), `CF_NORETURN`, `CF_LIKELY / CF_UNLIKELY` (branch hints), and `CF_BUILTIN_TRAP` (hard abort in debug builds).

2.3 Assertions

The macro `CF_ASSERT(cond, msg)` is active in debug builds (`CF_DEBUG` defined) and compiles to a no-op in release builds. When an assertion fires it calls `Caffeine::assertFailed`, which invokes `CF_BUILTIN_TRAP()` causing an immediate hardware trap. This is deliberately brutal: masked failures in debug are more dangerous than hard crashes.

Invariant 2.1 — Assert semantics No assertion shall be removed to silence a false positive. If `CF_ASSERT` fires, the precondition it guards must be fixed at the call site, not at the assertion.

2.4 Timing (`Timer.hpp`)

The `Timer` class wraps `std::chrono::high_resolution_clock` and exposes nanosecond-precision `TimePoint` values. The `Duration` struct carries time as a 64-bit `double` in seconds, with helper conversions to milliseconds, microseconds, and nanoseconds.

The `ScopeTimer` RAII guard is used for fine-grained profiling of editor sub-systems. Its destructor stops the timer and records the elapsed duration; the name string is a static pointer (not heap-allocated) to keep the measurement overhead negligible.

Chapter 3

Mathematics Library

3.1 Overview

The mathematics library in `src/math/` provides the complete set of primitive geometric types used throughout the engine. All types are plain structs with no virtual methods, no heap allocation, and scalar fields only — compatible with `memcpy` serialisation and SIMD reinterpretation on compilers that respect standard-layout guarantees.

3.2 Two-Dimensional Vectors (`Vec2`)

Definition 3.2.1 (2-Vector)**.** A two-dimensional vector $\mathbf{v} \in \mathbb{R}^2$ is defined as $\mathbf{v} = (x, y)$ where $x, y \in \mathbb{R}$.

The `Vec2` struct stores two `f32` components. The fundamental operations and their implementations are:

$$\text{Dot product:} \quad \mathbf{u} \cdot \mathbf{v} = u_x v_x + u_y v_y \tag{3.1}$$

$$\text{Squared length:} \quad |\mathbf{v}|^2 = \mathbf{v} \cdot \mathbf{v} \tag{3.2}$$

$$\text{Euclidean length:} \quad |\mathbf{v}| = \sqrt{|\mathbf{v}|^2} \tag{3.3}$$

$$\text{Normalisation:} \quad \hat{\mathbf{v}} = \begin{cases} \mathbf{v}/|\mathbf{v}| & \text{if } |\mathbf{v}| > 0 \\ \mathbf{0} & \text{otherwise} \end{cases} \tag{3.4}$$

The implementation guards against division by zero in both `length()` and `normalized()` by checking $|\mathbf{v}|^2 > 0$ before taking the square root or dividing.

3.3 Three-Dimensional Vectors (`Vec3`)

Definition 3.3.1 (3-Vector)**.** A three-dimensional vector $\mathbf{v} \in \mathbb{R}^3$ is $\mathbf{v} = (x, y, z)$ with components $x, y, z \in \mathbb{R}$.

In addition to the operations shared with `Vec2`, `Vec3` provides:

$$\text{Cross product:} \quad \mathbf{u} \times \mathbf{v} = \begin{pmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{pmatrix} \tag{3.5}$$

The cross product is anticommutative: $\mathbf{u} \times \mathbf{v} = -(\mathbf{v} \times \mathbf{u})$. It is used extensively in matrix construction (`lookAt`), Gram-Schmidt orthonormalisation, and quaternion rotation.

Static convenience accessors `Vec3::forward()`, `Vec3::up()`, and `Vec3::right()` return the canonical basis vectors $(0, 0, 1)$, $(0, 1, 0)$, and $(1, 0, 0)$ respectively.

3.4 Four-Dimensional Vectors (`Vec4`)

`Vec4` extends `Vec3` with a homogeneous w coordinate. It is used as the input to 4×4 matrix multiplications (points have $w = 1$; directions have $w = 0$) and as a colour representation (r, g, b, a).

3.5 4×4 Matrices (`Mat4`)

3.5.1 Storage Layout

`Mat4` stores 16 `f32` values in a flat array `m[16]` using *column-major* order. Element (row, col) maps to $\texttt{m}[col \cdot 4 + row]$. This layout is compatible with the OpenGL / Vulkan column-major convention and allows a matrix column to be loaded as a single 128-bit SIMD register.

3.5.2 Fundamental Transforms

Translation

$$\mathbf{T}(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Scale

$$\mathbf{S}(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation about Z, Y, X

$$\mathbf{R}_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \mathbf{R}_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Orthographic Projection

$$\mathbf{P}_{\text{ortho}} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

where l, r, b, t, n, f are the left, right, bottom, top, near, and far clipping planes.

Perspective Projection

Let θ_y be the vertical field-of-view angle and a the aspect ratio w/h.

$$\mathbf{P}_{\text{persp}} = \begin{pmatrix} \frac{1}{a\tan(\theta_y/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\theta_y/2)} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -1 \\ 0 & 0 & -\frac{2fn}{f-n} & 0 \end{pmatrix}$$

Look-At View Matrix

Given eye position $\mathbf{e}$, target $\mathbf{t}$, and world up $\mathbf{u}_w$:

$$\mathbf{f} = \widehat{\mathbf{t} - \mathbf{e}} \quad \text{(forward)} \tag{3.6}$$

$$\mathbf{r} = \widehat{\mathbf{f} \times \mathbf{u}_w} \quad \text{(right)} \tag{3.7}$$

$$\mathbf{u} = \mathbf{r} \times \mathbf{f} \quad \text{(corrected up)} \tag{3.8}$$

$$\mathbf{V} = \begin{pmatrix} r_x & r_y & r_z & -\mathbf{r}\cdot\mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{3.9}$$

3.5.3 Matrix Inversion

`Mat4::inverted()` implements the **cofactor expansion** method (Cramér's rule). The adjugate matrix $\text{adj}(\mathbf{M})$ is computed element-by-element as the transpose of the cofactor matrix. The inverse is:

$$\mathbf{M}^{-1} = \frac{1}{\det(\mathbf{M})} \cdot \text{adj}(\mathbf{M})$$

If $|\det(\mathbf{M})| < 10^{-6}$, the matrix is considered singular and the identity matrix is returned. This avoids undefined behaviour at the cost of silently returning an incorrect inverse; callers that construct degenerate matrices bear responsibility for that degeneracy.

3.6 Quaternions (`Quat`)

3.6.1 Mathematical Foundation

Definition 3.6.1 (Quaternion)**.** A quaternion $\amalg \in \mathbb{H}$ is an element of the form $\amalg = w + xi + yj + zk$ where $w, x, y, z \in \mathbb{R}$ and the imaginary units satisfy $i^2 = j^2 = k^2 = ijk = -1$ (Hamilton convention).

The engine stores quaternions as (x, y, z, w) in memory, where w is the scalar (real) part and (x, y, z) is the vector (imaginary) part $\amalg_v$.

A **unit quaternion** ($|\amalg| = 1$) represents a rotation. The rotation by angle θ around unit axis $\hat{\mathbf{n}}$ is:

$$\amalg = \left(\hat{\mathbf{n}} \sin\tfrac{\theta}{2},\ \cos\tfrac{\theta}{2}\right) \tag{3.10}$$

3.6.2 Quaternion Product (Hamilton Product)

$$\amalg_1 \cdot \amalg_2 = (w_1 w_2 - \amalg_{v1} \cdot \amalg_{v2},\ w_1 \amalg_{v2} + w_2 \amalg_{v1} + \amalg_{v1} \times \amalg_{v2}) \tag{3.11}$$

In component form (the implementation in `operator*`):

$$x' = w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 y_2 \tag{3.12}$$

$$y' = w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2 \tag{3.13}$$

$$z' = w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2 \tag{3.14}$$

$$w' = w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2 \tag{3.15}$$

The product $\amalg_1 \cdot \amalg_2$ applies $\amalg_2$ first, then $\amalg_1$ — the same composition order as matrix multiplication.

3.6.3 Vector Rotation

Rotating vector $\mathbf{v}$ by unit quaternion $\amalg$ is equivalent to $\amalg \cdot (0, \mathbf{v}) \cdot \amalg^{-1}$. The efficient formulation avoids the full product by exploiting the identity:

$$\mathbf{v}' = \mathbf{v} + 2\,\amalg_v \times (\amalg_v \times \mathbf{v} + w\,\mathbf{v}) \tag{3.16}$$

This requires only two cross products instead of two full quaternion products, reducing the operation from 28 multiplications to 15.

3.6.4 Quaternion to Rotation Matrix

For a unit quaternion $\amalg = (x, y, z, w)$:

$$\mathbf{R} = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - zw) & 2(xz + yw) \\ 2(xy + zw) & 1 - 2(x^2 + z^2) & 2(yz - xw) \\ 2(xz - yw) & 2(yz + xw) & 1 - 2(x^2 + y^2) \end{pmatrix}$$

This matrix satisfies $\mathbf{R}\mathbf{v} = \amalg \cdot \mathbf{v} \cdot \amalg^{-1}$ for any vector $\mathbf{v}$ and $\mathbf{R}\mathbf{R}^T = \mathbf{I}$ for unit quaternions.

3.6.5 Rotation Matrix to Quaternion (Shoemake 1987)

The trace-based algorithm of Ken Shoemake selects the numerically largest diagonal element to avoid division by near-zero values. Let $\text{tr} = R_{00} + R_{11} + R_{22}$.

Case $\text{tr} > 0$:

$$s = 2\sqrt{\text{tr} + 1}, \quad w = s/4, \quad x = (R_{21} - R_{12})/s, \quad y = (R_{02} - R_{20})/s, \quad z = (R_{10} - R_{01})/s$$

Case R_{00} is the largest diagonal:

$$s = 2\sqrt{1 + R_{00} - R_{11} - R_{22}}, \quad x = s/4, \quad y = (R_{01}+R_{10})/s, \quad z = (R_{02}+R_{20})/s, \quad w = (R_{21}-R_{12})/$$

Analogous formulae apply when R_{11} or R_{22} are largest.

3.6.6 Euler Angles (ZYX Tait-Bryan Convention)

The engine uses the ZYX (intrinsic) convention: roll (Z) applied first, then pitch (X), then yaw (Y) last. Given angles ϕ (roll), θ (pitch), ψ (yaw):

$$w = \cos(\phi/2)\cos(\psi/2)\cos(\theta/2) + \sin(\phi/2)\sin(\psi/2)\sin(\theta/2) \tag{3.17}$$
$$x = \cos(\phi/2)\cos(\psi/2)\sin(\theta/2) - \sin(\phi/2)\sin(\psi/2)\cos(\theta/2) \tag{3.18}$$
$$y = \cos(\phi/2)\sin(\psi/2)\cos(\theta/2) + \sin(\phi/2)\cos(\psi/2)\sin(\theta/2) \tag{3.19}$$
$$z = \sin(\phi/2)\cos(\psi/2)\cos(\theta/2) - \cos(\phi/2)\sin(\psi/2)\sin(\theta/2) \tag{3.20}$$

The inverse (quaternion to Euler) extracts angles from the rotation matrix rows via `atan2` and `asin`:

$$\psi = \arcsin\big(-2(xz - wy)\big) \quad \text{(yaw)} \tag{3.21}$$
$$\theta = \operatorname{atan2}\big(2(yz + wx),\, 1 - 2(x^2 + y^2)\big) \quad \text{(pitch)} \tag{3.22}$$
$$\phi = \operatorname{atan2}\big(2(xy + wz),\, 1 - 2(y^2 + z^2)\big) \quad \text{(roll)} \tag{3.23}$$

Gimbal lock occurs at $\psi = \pm\pi/2$ and is not resolved in the current implementation; users should prefer quaternions for continuous rotation.

3.6.7 Spherical Linear Interpolation (SLERP)

$$\text{SLERP}(\amalg_a, \amalg_b, t) = \frac{\sin((1-t)\Omega)}{\sin\Omega}\,\amalg_a + \frac{\sin(t\Omega)}{\sin\Omega}\,\amalg_b \tag{3.24}$$

where $\Omega = \arccos(|\amalg_a \cdot \amalg_b|)$. The shortest arc is guaranteed by negating $\amalg_b$ when $\amalg_a \cdot \amalg_b < 0$. When $\amalg_a \cdot \amalg_b > 0.9999$, linear interpolation (NLERP) is used to avoid the numerical instability of $\sin\Omega \approx 0$.

3.6.8 Normalised Linear Interpolation (NLERP)

$$\text{NLERP}(\amalg_a, \amalg_b, t) = \frac{(1-t)\,\amalg_a + t\,\amalg_b}{|(1-t)\,\amalg_a + t\,\amalg_b|} \tag{3.25}$$

NLERP does not maintain constant angular velocity — interpolation speed is faster near the midpoint — but is significantly cheaper (one normalisation vs. two `sinf`/`atan2f` evaluations).

3.7 Mathematical Utilities (`Math.hpp`)

The `Caffeine::Math` namespace provides scalar utility functions. Key values:

$$\pi = 3.14159265358979323846$$
$$\tau = 2\pi$$
$$e = 2.71828182845904523536$$

Smoothstep

The cubic Hermite interpolant used for smooth transitions:

$$\text{smoothstep}(e_0, e_1, x) = t^2(3-2t), \quad t = \text{saturate}\left(\frac{x-e_0}{e_1-e_0}\right) \tag{3.26}$$

Next Power of Two

```
usize nextPowerOfTwo(usize v) {
    --v;
    v |= v >> 1;  v |= v >> 2;
    v |= v >> 4;  v |= v >> 8;
    v |= v >> 16;
    return v + 1;
}
```

Listing 3.1: Bit-manipulation power-of-two rounding

This bit-spreading algorithm fills all lower bits of $v - 1$ with 1s, then adds 1. It runs in $O(\log_2 w)$ operations where w is the word width.

Chapter 4

Memory Management

4.1 The Allocator Interface (`IAllocator`)

All allocators implement the pure-virtual interface:

```
class IAllocator {
public:
    virtual void*  alloc(usize size, usize alignment = 8) = 0;
    virtual void   free(void* ptr) = 0;
    virtual void   reset() = 0;
    virtual usize  usedMemory()       const = 0;
    virtual usize  totalSize()        const = 0;
    virtual usize  peakMemory()       const = 0;
    virtual usize  allocationCount()  const = 0;
    virtual const char* name()        const = 0;
};
```

Listing 4.1: IAllocator interface

The alignment helper computes the number of padding bytes required to bring a pointer to the next alignment boundary:

$$\text{padding}(p, a) = \begin{cases} 0 & \text{if } p \bmod a = 0 \\ a - (p \bmod a) & \text{otherwise} \end{cases} \tag{4.1}$$

where a must be a power of two. The fast bitwise form is $(a - (p \,\&\, (a-1))) \,\&\, (a-1)$.

Invariant 4.1 — Alignment Every allocation returned by any `IAllocator` must satisfy: $(\text{address} \bmod \text{alignment}) = 0$. The engine assumes 8-byte minimum alignment by default.

4.2 Linear Allocator

Definition 4.2.1 (Linear Allocator)**.** A linear (or "bump-pointer") allocator maintains a single cursor c into a contiguous buffer $[s, s + N)$. Allocation advances the cursor: $c' = c + \text{padding}(c, a) + \text{size}$. The only valid "free" operation is a full reset: $c \leftarrow s$.

Complexity: $O(1)$ allocation, $O(1)$ reset. No per-allocation overhead beyond alignment padding.

Use cases: per-frame scratch memory, loading screens, temporary computation buffers. The pattern is: *allocate everything, process, reset at frame end*. No individual deallocations are ever needed.

[scale=0.9] [thick] (0,0) rectangle (8,0.6); [darkblue!20] (0,0) rectangle (4.5,0.6); [darkblue, thick, ->] (4.5,0.8) – (4.5,0.6) node[above, yshift=4pt]cursor; at (2.25,0.3) used; at (6.25,0.3) free;

Figure 4.1: Linear allocator state: the cursor divides the buffer into used and free regions.

4.3 Pool Allocator

Definition 4.3.1 (Pool Allocator)**.** A pool allocator divides a buffer into N fixed-size *slots* of size s. Free slots are linked in a *free-list*: each free slot stores a pointer to the next free slot. Allocation pops the head of the list; deallocation pushes back onto the head.

Complexity: $O(1)$ allocation, $O(1)$ deallocation. The list is embedded in-place: no separate bookkeeping array.

The free-list is initialised in reverse order so that the first allocation returns the slot at the lowest address:

```
for (usize i = 0; i < m_maxSlots; ++i) {
    u8* slot = m_poolStart + i * m_slotSize;
    *reinterpret_cast<u8**>(slot) = m_freeList;
    m_freeList = slot;
}
```

Listing 4.2: Free-list initialisation (reversed)

Use cases: ECS component pools, audio source instances, particle systems — any subsystem that creates and destroys many identically-sized objects at high frequency.

Invariant 4.2 — Slot size No allocation request may exceed `m_slotSize`. The pool does not track individual allocation sizes; a larger request would corrupt adjacent slots.

4.4 Stack Allocator

The stack allocator is structurally identical to the linear allocator but adds the concept of a *Marker*:

```
Marker m = stack.setMarker();     // snapshot cursor
// ... temporaries ...
stack.freeToMarker(m);            // roll back to m
```

Listing 4.3: Marker-based rollback

A marker is simply a byte offset from the buffer start (`usize`). `freeToMarker` moves the cursor back to the saved offset, effectively deallocating everything allocated since the marker was set. This supports LIFO (last-in, first-out) deallocation at $O(1)$ cost.

Use cases: recursive descent parsing, multi-phase asset loading where each phase's scratch memory must be freed before the next phase.

Chapter 5

Container Library

5.1 Dynamic Array (`Vector<T>`)

`Vector<T>` is a heap-growing array equivalent to `std::vector`, but with `IAllocator` injection. When no allocator is provided, `operator new` / `operator delete` are used directly.

Growth Strategy

$$\text{newCapacity} = \begin{cases} 8 & \text{if current capacity} = 0 \\ 2 \times \text{current capacity} & \text{otherwise} \end{cases} \tag{5.1}$$

Doubling maintains amortised $O(1)$ push-back: a sequence of n push-backs performs at most $2n$ element moves total.

Construction and Destruction

Elements are constructed in-place via placement-`new`: `new (&m_data[m_size]) T(value)`. They are explicitly destroyed before deallocation: `m_data[i].~T()`. This ensures correct behaviour for non-trivial types while remaining compatible with the custom allocator interface.

Move Semantics

The move constructor and move assignment transfer ownership of the underlying buffer in $O(1)$ by pointer swap, leaving the source in a valid empty state.

5.2 Hash Map (`HashMap<K,V>`)

The current `HashMap` is a *linear-scan* map backed by a `Vector<Pair>`. All operations are $O(n)$ in the number of entries.

Remark 5.2.1. The current implementation is intentionally simple: the expected use-cases (editor panel registries, asset name→ID tables) have at most a few hundred entries where cache-friendly linear scan outperforms hash table overhead. A true hash table (open addressing, Robin-Hood probing) is planned for `v2.0` when scene complexity requires it.

5.3 String View (`StringView`)

`StringView` is a non-owning, read-only view into a null-terminated string: a pointer + length pair. It performs no heap allocation. Comparison is lexicographic and runs in $O(\min(|a|, |b|))$.

5.4 Fixed String (`FixedString<N>`)

`FixedString<N>` stores at most $N - 1$ characters in an inline `char[N]` array plus a length field. There is no heap involvement. It is used for entity names, asset paths, and any string that must be embeddable inside a struct without pointer indirection.

Chapter 6

Entity-Component System (ECS)

6.1 Design Philosophy

The ECS in Caffeine is *archetype-based*: entities with the same set of component types share a single `Archetype`. This groups the data for all entities with a given composition contiguously in memory, enabling iteration over a specific component type to proceed without cache misses.

Definition 6.1.1 (Entity)**.** An entity is an unsigned 32-bit integer identifier. It carries no data. All observable state is stored in components associated with the entity. An entity with index `u32_max` is conventionally *invalid*.

Definition 6.1.2 (Component)**.** A component is a plain-old-data (POD) struct. It must have no virtual methods and no hidden heap allocation. The engine enforces this by instantiating components through typed `ComponentPool<T>` which stores them in a `Vector<T>`.

Definition 6.1.3 (Archetype)**.** An archetype $\mathcal{A}$ is the set of component types associated with a group of entities. Two entities belong to the same archetype if and only if they possess exactly the same set of component types.

6.2 Component Identification

Each component type receives a unique 32-bit ID at program initialisation via `ComponentID::get<T>`.

```
template<typename T>
static u32 get() {
    static const u32 id = nextID();  // initialised once per type
    return id;
}
```

Listing 6.1: Type-ID registration

The `static` local variable is guaranteed to be initialised exactly once (C++11 magic statics). The counter is an `std::atomic<u32>` to support concurrent registration from multiple translation units. Up to 256 distinct component types are permitted.

6.3 Component Set (`ComponentSet`)

A `ComponentSet` is a bitmask of component IDs. Archetype identity is determined by comparing two `ComponentSet`s. Set operations (union, intersection, subset test) reduce to bitwise OR, AND, and masking.

6.4 Archetype Internal Structure

Each `Archetype` contains:

- A `Vector<u32>` of entity IDs (the entities belonging to it).
- A `PoolRegistry`: a fixed-size array of 64 pool entries, indexed by component ID, each entry holding a `void*` pool pointer and function pointers for copy, create, and remove.

Removal by Swap-and-Pop

When an entity at index i is removed from an archetype, the entity at the last position $n-1$ is moved into slot i. This preserves contiguity in $O(1)$ without shifting all subsequent elements.

$$\text{entity}[i] \leftarrow \text{entity}[n-1]; \quad n \leftarrow n-1 \tag{6.1}$$

The same swap-and-pop is applied to every component pool for that archetype.

6.5 Command Buffer

Structural mutations (creating or destroying entities, adding or removing components) *during iteration* would invalidate in-flight iterators. The `CommandBuffer` defers all mutations by recording them as `ICommand` objects:

```
Entity CommandBuffer::create() {
    u32 pendingID = m_nextPendingID++;
    m_commands.pushBack(make_unique<CreateCommand>(pendingID));
    return Entity(pendingID, nullptr);  // not yet alive in world
}
```

Listing 6.2: Deferred entity creation

`execute(World&)` is called at a safe point (end of frame, after all systems finish) to replay all recorded mutations on the live world.

Invariant 6.1 — Structural mutation safety ECS structural mutations (entity creation/destruction, component add/remove) during iteration **must** go through `CommandBuffer`. Direct world mutation during a `forEach` loop is undefined behaviour.

6.6 Systems

Systems implement `ISystem` and are registered in `SystemRegistry`. Each system exposes `onUpdate(World&, f32 dt)`. The world provides `forEach<C1, C2, ...>` which iterates over all entities possessing the queried component types, invoking a callable with typed references.

6.7 Component Queries

The `ComponentQuery` class provides a builder interface for selecting archetypes based on component presence, absence, or partial matches. It maintains three criteria:

- `m_required`: A `ComponentSet` of types that **must** be present.
- `m_excluded`: A `ComponentSet` of types that **must not** be present.
- `m_any`: A vector of `ComponentSet`s, where for each set, **at least one** component must be present.

The builder methods `with<T>()`, `without<T>()`, and `any<T...>()` populate these masks using `ComponentID::get<T>()`.

6.7.1 Matching Logic

Given an archetype's component set $\mathcal{A}$, the query matches if and only if the following boolean expression is true:

$$(\mathcal{R} \subseteq \mathcal{A}) \wedge (\mathcal{E} \cap \mathcal{A} = \emptyset) \wedge (\forall \mathcal{S} \in \mathcal{M}_{any} : \mathcal{S} \cap \mathcal{A} \neq \emptyset) \tag{6.2}$$

where $\mathcal{R}$ is the required set, $\mathcal{E}$ is the excluded set, and $\mathcal{M}_{any}$ is the collection of "any" sets.

```
ComponentQuery query;
query.with<Transform, Mesh>()
     .without<Hidden>()
     .any<Sprite, ParticleEmitter>();

if (query.matches(someArchetype)) {
    // Process archetype...
}
```

Listing 6.3: ComponentQuery API usage

Chapter 7

Job System and Concurrency

7.1 Architecture Overview

The job system is a **work-stealing thread pool** with three priority levels: `Critical`, `Normal`, and `Background`. Each worker thread has a local double-ended queue (deque) per priority level, plus a set of global overflow queues.

[scale=0.85, every node/.style=font=] ıin 0,1,2 [thick] (3.5*ı, 0) rectangle (3.5*ı+2.5, 1.2); at (3.5*ı+1.25, 0.6) Worker ı; [thick, darkblue] (3.5*ı, -0.3) rectangle (3.5*ı+2.5, -1.5); [darkblue] at (3.5*ı+1.25, -0.9) Local deque; [thick, accentblue] (0, -2.2) rectangle (9.5, -3.2); [accentblue] at (4.75, -2.7) Global queues (Critical / Normal / Background); ıin 0,1,2 [->] (3.5*ı+1.25,-1.5) – (3.5*ı+1.25,-2.2); [->, dashed] (3.5*ı+2.5,-0.9) to[bend right=15] (3.5*ı+3.5+1.25,-0.9);

Figure 7.1: Job system topology: each worker has private local queues; dashed arrows indicate work-stealing.

7.2 Work-Stealing Protocol

On each worker tick, the following priority order is applied:

1. Pop from own local queue (Critical first, then Normal, then Background).
2. Pop from the global queue at each priority level.
3. **Steal** from another worker's local queue (round-robin, from the *back* of the victim's deque to minimise contention).

Stealing from the back of the victim and consuming from the front of self's deque is the classic Chase-Lev deque design. This minimises contention: the owner operates on the front; stealers operate on the back.

7.3 Job Handles and the ABA Problem

Each job is assigned a slot index and a version number. The handle stores both. A handle is considered complete when `m_slots[index].flag == 1 AND m_slots[index].version == handle.version`. Reusing the slot for a different job increments its version, preventing a stale handle from falsely reporting completion.

Invariant 7.1 — Handle version check Job handles must compare both slot index *and* version number before reporting completion. Checking only the index is susceptible to ABA-style races where a recycled slot appears done prematurely.

7.4 Barriers

`JobBarrier` is a synchronisation point: a job may call `barrier.release()` upon completion; a caller blocks on `barrier.wait()` until the count reaches zero. The barrier uses a `std::condition_variable` to avoid busy-waiting.

7.5 Parallel For

`scheduleParallelFor(count, func)` divides $[0, \text{count})$ into $\min(\text{workerCount}, \text{count})$ chunks. Each chunk $[b_i, e_i)$ is scheduled as an independent job. A shared atomic counter tracks how many chunks have completed; the last chunk to decrement it to zero signals the parent handle:

$$\text{chunkSize}_i = \left\lfloor \frac{N}{k} \right\rfloor + [i < N \bmod k] \tag{7.1}$$

where N is the total count and k is the number of chunks.

7.6 Worker Count

If the caller passes 0, the system uses `hardware_concurrency()` $- 1$ workers, leaving one logical core for the main thread.

Chapter 8

2D Physics System

8.1 Components

8.1.1 RigidBody2D

Stores dynamic properties: `mass` m, `restitution` e, `friction` μ, `linearDamping` d, flags `isKinematic` and `isSleeping`.

8.1.2 Collider2D

Stores geometric properties: `shape` (AABB or Circle), `size/radius`, `offset`, collision `layer` / `layerMask` bitmask, and flags `isStatic`, `isTrigger`, `isOneWay`.

8.2 Simulation Loop

The system runs $k = 2$ sub-steps per frame ($k =$ `kSubSteps`):

$$\Delta t_{\text{sub}} = \frac{\Delta t}{k} \tag{8.1}$$

Per sub-step:

1. Apply queued forces and impulses.
2. Integrate velocities and positions (semi-implicit Euler).
3. Build the spatial hash grid.
4. Detect and resolve collisions.
5. Update sleep state.

8.3 Integration (Semi-Implicit Euler)

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{g}\,\Delta t_{\text{sub}} \quad \text{(gravity accumulation)} \tag{8.2}$$

$$\mathbf{v}_{n+1} \leftarrow \mathbf{v}_{n+1} \cdot \max(0,\, 1 - d\,\Delta t_{\text{sub}}) \quad \text{(linear damping)} \tag{8.3}$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+1}\,\Delta t_{\text{sub}} \tag{8.4}$$

The velocity is updated *before* the position (semi-implicit), which is more stable than explicit Euler for spring-like constraints.

8.4 Broad-Phase: Uniform Grid

The spatial hash maps a 2D integer cell coordinate (c_x, c_y) to a list of entity IDs. The cell coordinate of a world position p is:

$$c = \left\lfloor \frac{p}{\texttt{kGridCellSize}} \right\rfloor \tag{8.5}$$

with $\texttt{kGridCellSize} = 128$ world units. An entity whose AABB spans multiple cells is inserted into all cells it touches. The cell key is computed as:

$$\text{key}(c_x, c_y) = c_x \cdot 73856093 \oplus c_y \cdot 19349663 \tag{8.6}$$

(a standard spatial hashing polynomial). Candidate collision pairs are generated by checking all pairs within the same cell, with duplicate suppression.

8.5 Narrow-Phase Geometry

8.5.1 AABB vs. AABB

Let centres be $\mathbf{p}_A, \mathbf{p}_B$ with half-extents $\mathbf{h}_A, \mathbf{h}_B$. A collision occurs when:

$$\text{overlapX} = (h_{Ax} + h_{Bx}) - |p_{Bx} - p_{Ax}| > 0 \tag{8.7}$$

$$\text{overlapY} = (h_{Ay} + h_{By}) - |p_{By} - p_{Ay}| > 0 \tag{8.8}$$

The separation axis is the one with *minimum overlap*:

$$\mathbf{n} = \begin{cases} (\text{sgn}(\Delta x),\, 0) & \text{if overlapX} < \text{overlapY} \\ (0,\, \text{sgn}(\Delta y)) & \text{otherwise} \end{cases} \tag{8.9}$$

8.5.2 Circle vs. Circle

Let distance $d = |\mathbf{p}_B - \mathbf{p}_A|$ and sum of radii $r = r_A + r_B$.

$$\text{collision} \Leftrightarrow d < r \tag{8.10}$$

$$\mathbf{n} = (\mathbf{p}_B - \mathbf{p}_A)/d \tag{8.11}$$

$$\text{penetration} = r - d \tag{8.12}$$

8.5.3 Circle vs. AABB

The closest point on the AABB to the circle centre is:

$$\mathbf{c} = \text{clamp}(\mathbf{p}_{\text{circle}},\, \mathbf{p}_{\text{AABB}} - \mathbf{h},\, \mathbf{p}_{\text{AABB}} + \mathbf{h}) \tag{8.13}$$

If $|\mathbf{p}_{\text{circle}} - \mathbf{c}| < r$, a collision is detected.

8.6 Impulse-Based Resolution

Let relative velocity along the collision normal be:

$$v_{\text{rel}} = (\mathbf{v}_B - \mathbf{v}_A) \cdot \mathbf{n} \tag{8.14}$$

If $v_{\text{rel}} > 0$ (separating), no impulse is applied. Otherwise, the scalar impulse magnitude is:

$$j = \frac{-(1+e)\, v_{\text{rel}}}{m_A^{-1} + m_B^{-1}} \tag{8.15}$$

where $e = \min(e_A, e_B)$ is the coefficient of restitution. Velocities are updated:

$$\mathbf{v}_A \leftarrow \mathbf{v}_A - j\, m_A^{-1}\, \mathbf{n} \tag{8.16}$$

$$\mathbf{v}_B \leftarrow \mathbf{v}_B + j\, m_B^{-1}\, \mathbf{n} \tag{8.17}$$

Friction

The tangential impulse j_t is:

$$j_t = \frac{-v_{\text{rel,tan}}}{m_A^{-1} + m_B^{-1}}, \quad \mu = \sqrt{\mu_A \mu_B} \tag{8.18}$$

Applied as a Coulomb friction cone: $|j_t| \leq \mu |j|$.

8.7 Positional Correction (Baumgarte)

To prevent sinking, the positions are corrected proportionally to the penetration depth, with a slop tolerance δ_{slop} and factor k_{B}:

$$\Delta \mathbf{x} = \frac{(\text{penetration} - \delta_{\text{slop}}) \cdot k_{\text{B}}}{m_A^{-1} + m_B^{-1}} \, \mathbf{n} \tag{8.19}$$

with $\delta_{\text{slop}} = 0.01$ and $k_{\text{B}} = 0.4$.

8.8 Sleep System

An entity enters sleep when its velocity magnitude drops below $v_{\text{sleep}} = 0.05$ for longer than $t_{\text{sleep}} = 0.5\,\text{s}$. A sleeping entity skips integration entirely, saving compute for static scenes with many resting bodies. Any external force or impulse wakes the entity immediately.

8.9 Raycast

A ray $\mathbf{r}(t) = \mathbf{o} + t\hat{\mathbf{d}}$ is tested against all entities with a `Collider2D`. For AABB colliders the slab method is used; for circle colliders the quadratic form:

$$|(\mathbf{o} + t\hat{\mathbf{d}}) - \mathbf{c}|^2 = r^2 \tag{8.20}$$

is solved for t. The hit with minimum $t \geq 0$ is returned.

Chapter 9

Spatial Audio System

9.1 Architecture

The audio system wraps SDL3's `SDL_AudioStream` API. It maintains a pool of `AudioSource` objects (default 32 SFX channels). Each source maps to one `SDL_AudioStream` bound to the system's `SDL_AudioDeviceID` (default device, S16 stereo 44100 Hz).

9.2 AudioClip

An `AudioClip` is a pure descriptor: a pointer to raw PCM bytes, size, sample rate, channel count, bits per sample, and duration. The system does not own the memory; clips are registered by the asset pipeline which manages their lifetimes.

9.3 Spatial Attenuation

For an emitter at world position $\mathbf{p}_e$ and listener at $\mathbf{p}_l$ with maximum audible distance d_{max}:

$$\text{volume} = \max\left(0,\ 1 - \frac{|\mathbf{p}_e - \mathbf{p}_l|}{d_{\text{max}}}\right) \tag{9.1}$$

This is a *linear falloff* model. Physically accurate inverse-square falloff ($1/d^2$) sounds too aggressive for game contexts; linear falloff produces a smoother perceptual transition.

9.4 Stereo Panning

The pan value in $[-1, 1]$ is derived from the horizontal offset of the emitter relative to the listener:

$$\text{pan} = \text{clamp}\left(\frac{p_{ex} - p_{lx}}{d_{\text{max}}},\ -1,\ 1\right) \tag{9.2}$$

Gain for each channel:

$$g_L = v \cdot (1 - \max(0, \text{pan})) \tag{9.3}$$

$$g_R = v \cdot (1 + \min(0, \text{pan})) \tag{9.4}$$

The stream gain is set to $\max(g_L, g_R)$. True per-channel gain requires a custom mixing callback (planned).

9.5 Playback Loop

When a looping source exhausts its buffer (SDL reports `SDL_GetAudioStreamAvailable` == 0), the full clip data is re-submitted via `SDL_PutAudioStreamData`. The implementation tracks elapsed time to detect this condition frame-accurately.

Chapter 10

Asset Pipeline (.caf Format)

10.1 Design Goals

The Caffeine Asset Format (`.caf`) is a **zero-parsing, zero-copy** binary container. The layout on disk is a direct mirror of the in-RAM / in-VRAM layout, so loading is a single `fread` followed by pointer arithmetic — no deserialization step, no allocations beyond the buffer itself.

10.2 File Layout

$$\underbrace{[0\ldots31]}_{\text{CafHeader}}\underbrace{[32\ldots32+N-1]}_{\text{Metadata}}\underbrace{[32+N\ldots32+N+M-1]}_{\text{Payload}}\underbrace{[32+N+M\ldots32+N+M+3]}_{\text{Footer CRC32}}$$

Table 10.1: CafHeader fields (32 bytes, little-endian)

Offset	Field	Type	Bytes
0	`magic`	`u32` (= 0xCAFECAFE)	4
4	`versionMajor`	`u16`	2
6	`versionMinor`	`u16`	2
8	`type`	`AssetType` (u16)	2
10	`flags`	`CafFlags` (u16)	2
12	`crc32`	`u32`	4
16	`metadataSize`	`u64`	8
24	`dataSize`	`u64`	8

Invariant 10.1 — Endianness The `.caf` format is little-endian. A `static_assert` verifies `std::endian::native == std::endian::little` at compile time; the format must not be used on big-endian platforms without byte-swapping shims.

10.3 Integrity Verification

Two CRC32 checks are performed at load time:

1. **Payload CRC32**: `crc32(payload, dataSize)` must equal `header.crc32`.
2. **Whole-file CRC32**: The 4-byte footer stores `crc32(everything-before-footer)`, providing end-to-end integrity against file corruption or truncation.

The CRC32 uses the IEEE 802.3 polynomial $P =$ `0xEDB88320` (reflected form). The lookup table of 256 entries is built at compile time via a `constexpr` function, adding zero run-time initialisation cost.

Invariant 10.2 — CRC32 verification Both CRC32 checks must pass before any `.caf` file is used. A file that passes only one check is treated as corrupt.

10.4 Asset Types

Table 10.2: AssetType discriminants and their metadata structs

Type	Metadata struct	Notes
`Texture`	`TextureMetadata` (24 B)	width, height, format, mips
`Audio`	`AudioMetadata` (16 B)	sampleRate, channels, samples
`Mesh`	(Phase 5)	vertex / index buffers
`Prefab`	`SceneMetadata` (20 B)	binary ECS entity template
`Scene`	`SceneMetadata` (20 B)	full world state snapshot
`Shader`	`ShaderMetadata` (8 B)	stage (vert/frag/compute)
`Animation`	(planned)	animation clip frames

10.5 Live Asset Watching

`FileWatcher` monitors a directory via `ReadDirectoryChangesW` (Windows) or a polling stub (other platforms). Changed paths are pushed onto a thread-safe queue and polled from the main thread so that the asset loader can hot-reload without cross-thread world mutation.

10.6 Runtime Asset Types

While metadata defines the disk layout, runtime code interacts with zero-copy view types. These structures do not own memory; they hold pointers directly into the `LinearAllocator` buffer where the `.caf` payload was loaded.

- **Texture**: Contains `width`, `height`, `format`, and `mipLevels`. The `pixels` pointer provides raw access to the texel data of size `pixelDataSize`.
- **AudioClip**: Encapsulates PCM data with `sampleRate`, `channels`, `bitsPerSample`, and `sampleCount`. Raw data is accessed via `pcmData`.
- **ShaderBlob**: A wrapper for compiled bytecode (`bytecode`, `bytecodeSize`) and its associated pipeline `stage` (Vertex, Fragment, or Compute).

10.6.1 Asset Type Traits

The `AssetTypeTrait<T>` template specialization mechanism is used to map C++ runtime types to their corresponding `AssetType` discriminants. This allows the `AssetManager` to perform type-safe loading and caching using the `.caf` header information.

```
template<> struct AssetTypeTrait<Texture> {
    static constexpr AssetType cafType = AssetType::Texture;
};
```

Listing 10.1: AssetTypeTrait specialization

Chapter 11

Game Loop

11.1 Fixed Timestep with Interpolation

The game loop implements a **fixed-timestep with remainder interpolation** — the pattern described by Glenn Fiedler in “Fix Your Timestep” (2004).

```
accumulator += min(deltaTime, maxFrameTime);
while (accumulator >= fixedDeltaTime) {
    fixedUpdate(fixedDeltaTime);
    accumulator -= fixedDeltaTime;
}
alpha = accumulator / fixedDeltaTime;
render(alpha);
```

Listing 11.1: Game loop tick

Spiral of Death Protection

Clamping Δt to `maxFrameTime` (default $0.25\,\text{s}$) prevents the *spiral of death*: if a frame takes longer than Δt_{fixed}, the accumulator would grow without bound, scheduling infinitely many fixed-update steps, making the frame even longer. The clamp trades simulation accuracy (the simulation slows down in real time) for stability.

Interpolation Alpha

$\alpha = \texttt{accumulator}/\Delta t_{\text{fixed}} \in [0, 1)$ is the fraction of a fixed step that has elapsed but not yet been simulated. The renderer uses it to interpolate between the previous and current physics state for sub-frame smooth visual output.

11.2 Debug Hook Registry

`DebugHookRegistry` provides a single-slot registration for an `IDebugHooks` implementation. The core engine calls hooks unconditionally-if-registered:

```
if (auto* h = DebugHookRegistry::hooks()) {
    h->onFrameEnd(stats);
}
```

Listing 11.2: Zero-overhead hook call pattern

This decouples the core from the editor without requiring a virtual call on every path: when no hooks are registered (shipped game), the branch is predicted-not-taken with negligible cost.

Chapter 12

Editor and Scene Viewport

12.1 Overview

The scene viewport (`SceneViewport`) is a 100% software-rendered panel driven by ImGui's `ImDrawList`. There is no GPU projection matrix; all world-to-screen transformations are implemented analytically in the `projectToScreen` function. Three view modes are supported: **Mode2D** (orthographic top-down), **Mode3D** (perspective orbit), and **Isometric** (dimetric projection).

12.2 Camera Model

The camera state is stored in `EditorContext`:

Table 12.1: Camera state variables

Variable	Type	Meaning
`camYaw`	`f32`	Azimuth angle (radians, Y-axis)
`camPitch`	`f32`	Elevation angle (radians, X-axis), clamped to $[-\pi/2, \pi/2]$
`camFocus`	`Vec3`	World-space point the camera orbits
`camDistance`	`f32`	Distance from focus to eye

12.3 projectToScreen — Mode2D

In 2D mode, the projection is a simple affine pan-and-zoom:

$$s_x = \text{origin}_x + \frac{W}{2} + p_x \cdot \text{zoom} + \text{panX} \tag{12.1}$$

$$s_y = \text{origin}_y + \frac{H}{2} - p_y \cdot \text{zoom} + \text{panY} \tag{12.2}$$

where W, H are the viewport dimensions and (p_x, p_y) is the world position.

12.4 projectToScreen — Mode3D

The 3D projection is a software perspective transform using the camera's spherical parameterisation.

Step 1: Translate to Camera-Relative Space

$$\mathbf{r} = \mathbf{p} - \mathbf{f} \tag{12.3}$$

where $\mathbf{f} = \texttt{camFocus}$.

Step 2: Yaw Rotation (Y-axis, angle ψ)

$$v_x = \cos\psi \cdot r_x + \sin\psi \cdot r_z \tag{12.4}$$
$$v_y = r_y \tag{12.5}$$
$$v_z = -\sin\psi \cdot r_x + \cos\psi \cdot r_z \tag{12.6}$$

The yaw rotation brings the camera's viewing direction along the $+Z$ axis in view space.

Step 3: Pitch Rotation (X-axis, angle ϕ)

$$v_{y2} = \cos\phi \cdot v_y + \sin\phi \cdot v_z \tag{12.7}$$
$$v_{z2} = -\sin\phi \cdot v_y + \cos\phi \cdot v_z \tag{12.8}$$

Step 4: Perspective Divide

The viewing volume is parameterised by `camDistance` D:

$$\text{fovScale} = \frac{s \cdot D}{\max(D + v_{z2},\ \varepsilon)} \tag{12.9}$$

where $s = \min(W, H)/2$ is the half-screen size used as a scale factor, and $\varepsilon = 0.01$ guards against division by zero. The screen coordinates are:

$$s_x = c_x + v_x \cdot \text{fovScale} + \text{panX} \tag{12.10}$$
$$s_y = c_y - v_{y2} \cdot \text{fovScale} + \text{panY} \tag{12.11}$$

Near-Plane Clipping

A point is *behind the camera* when $D + v_{z2} \leq \varepsilon$. For line segments (grid, gizmo axes), this causes `fovScale` to approach infinity, projecting to extreme screen positions. The fix is near-plane clipping: for a segment $(\mathbf{a}, \mathbf{b})$, compute the camera-depth of each endpoint d_A, d_B. If exactly one endpoint is behind the near plane (depth $< \varepsilon$), linearly interpolate the segment to the boundary:

$$t = \frac{\varepsilon - d_A}{d_B - d_A}, \quad \mathbf{c} = \mathbf{a} + t(\mathbf{b} - \mathbf{a}) \tag{12.12}$$

The clipped segment $(\mathbf{c}, \mathbf{b})$ (or $(\mathbf{a}, \mathbf{c})$) is then safe to project without numerical explosion.

Invariant 12.1 — Near-plane clipping Near-plane clipping must be applied to every projected line segment in Mode3D. Skipping it for "short" segments is not safe; the perspective divide can still explode for world-space points near the camera position.

12.5 projectToScreen — Isometric

The isometric projection uses a fixed dimetric angle offset of $\alpha_0 = 30^\circ = \pi/6$ added to the azimuth:

$$\cos A = \cos(\psi + \alpha_0) \quad \text{(effective azimuth)} \tag{12.13}$$

$$\sin A = \sin(\psi + \alpha_0) \tag{12.14}$$

$$s_x = c_x + (r_x \cdot \cos A + r_z \cdot \sin A) \cdot \text{zoom} + \text{panX} \tag{12.15}$$

$$s_y = c_y - (r_y - r_x \cdot \sin A \cdot 0.5 + r_z \cdot \cos A \cdot 0.5) \cdot \text{zoom} + \text{panY} \tag{12.16}$$

This produces the classic isometric look without using a GPU depth buffer.

12.6 Infinite Grid Rendering

The 3D grid is drawn on the XZ plane ($y = 0$). Grid lines are world segments of the form:

$$\{(i, 0, z) : z \in [-H, H]\} \quad \text{(X-aligned lines)} \tag{12.17}$$

$$\{(x, 0, i) : x \in [-H, H]\} \quad \text{(Z-aligned lines)} \tag{12.18}$$

where $i \in \{-H_{\text{lines}}, \ldots, H_{\text{lines}}\}$ and H_{lines} is chosen dynamically based on `camDistance`.

Each segment is near-plane clipped before projection. The adaptive spacing doubles when the grid density exceeds 60 lines on screen:

$$\text{spacing} \leftarrow 2 \cdot \text{spacing} \quad \text{while } \frac{2 \cdot H_{\text{lines}}}{\text{spacing}} > 60 \tag{12.19}$$

12.7 Transform Gizmo

The `TransformGizmo` renders translate, rotate, and scale handles directly on the `ImDrawList`. For each axis $A \in \{X, Y, Z\}$, the tip position is computed by projecting the world point $\mathbf{p} + \Delta A \cdot \hat{\mathbf{e}}_A$ through `projectToScreen`.

Axis Normalisation

To keep handles at a constant screen-space length L (pixel units), the raw projected tip is normalised:

$$\text{end}_A = \mathbf{o} + L \cdot \frac{\text{rawEnd}_A - \mathbf{o}}{|\text{rawEnd}_A - \mathbf{o}|} \tag{12.20}$$

If $|\text{rawEnd}_A - \mathbf{o}| < 3$ px (the axis projects nearly onto the screen-space origin), a per-axis fallback direction is used.

Z-Axis Overlap Guard

When the Z-axis fallback direction projects to within $0.3 \cdot L$ pixels of the Y-axis endpoint, the Z-axis is forced to its default fallback $(-0.6L, +0.6L)$ to avoid ambiguity:

$$d_{ZY} = |\text{end}_Z - \text{end}_Y| < 0.3L \implies \text{end}_Z \leftarrow \text{fallback}_Z \tag{12.21}$$

12.8 Navigation Widget (Orientation Gizmo)

The mini orientation gizmo in the viewport corner shows the camera's current viewing direction. Each world axis is projected onto screen space using the camera rotation only (no perspective divide):

$$s_x = \cos\psi \cdot A_x + \sin\psi \cdot A_z \tag{12.22}$$

$$s_{y1} = A_y \tag{12.23}$$

$$s_z = -\sin\psi \cdot A_x + \cos\psi \cdot A_z \tag{12.24}$$

$$s_{y2} = \cos\phi \cdot s_{y1} + \sin\phi \cdot s_z \tag{12.25}$$

$$\text{dir} = \left(\frac{s_x}{L},\ \frac{-s_{y2}}{L}\right) \cdot L_{\text{widget}} \tag{12.26}$$

where (A_x, A_y, A_z) is the world-space direction of the axis (e.g. $(1, 0, 0)$ for X), ψ and ϕ are `camYaw` and `camPitch`, and $L_{\text{widget}} = 22$ px is the display length.

The three axes are drawn in fixed colours: X red (255, 70, 70), Y green (70, 255, 90), Z blue (90, 140, 255).

Invariant 12.2 — Navigation widget correctness Navigation widget axis directions must be computed from the live `camYaw` and `camPitch` values every frame. Hardcoding or caching the directions between frames is incorrect.

12.9 Keyboard Navigation

Arrow key navigation moves `camFocus` in the camera's horizontal plane (pitch is ignored for navigation to avoid tilting the focus point out of the ground plane):

$$\Delta\mathbf{f}_{\text{forward}} = (-\sin\psi,\ 0,\ \cos\psi) \cdot v \tag{12.27}$$

$$\Delta\mathbf{f}_{\text{right}} = (\cos\psi,\ 0,\ \sin\psi) \cdot v \tag{12.28}$$

The `ImGuiWindowFlags_NoNavInputs` flag on the viewport window prevents ImGui from consuming arrow key events to navigate UI buttons.

Chapter 13

Editor Architecture and Undo System

13.1 EditorContext

`EditorContext` is the singleton shared state passed to every panel. It holds selection state, gizmo mode, snap settings, the camera parameters described in Chapter 12, and the `UndoStack`.

13.2 UndoStack

The undo system uses **full world snapshots**: each `EditorCommand` stores a `beforeState` and `afterState` as `std::vector<u8>` binary blobs. Undo restores the before state; redo restores the after state.

The ring buffer holds up to 256 commands. A new push after an undo truncates the redo history (linear undo). This is the simplest correct implementation; a command-specific delta approach (e.g. storing only the modified component fields) would be more memory-efficient but requires serialisation round-trips for every component type.

13.3 Panel System

Each editor panel is an autonomous class with an `onImGuiRender(World&, EditorContext&)` method: `HierarchyPanel`, `InspectorPanel`, `AssetBrowser`, `AnimationTimeline`, `AudioPreviewPan` `BuildDialog`, etc. Panels communicate solely through `EditorContext`; no panel holds a pointer to another panel.

Chapter 14

Implementation Rules and Future Specifications

14.1 Binding Invariants

The following invariants must be preserved by all future implementations. A proposed change that violates any invariant requires explicit discussion and this document must be updated before implementation begins.

Table 14.1: Binding invariants summary

ID	Statement
I-1	All type widths validated by `static_assert`.
I-2	`CF_ASSERT` is never removed to silence errors.
I-3	Allocator alignment is always a power of two ≥ 8.
I-4	Pool slot size is never exceeded at the call site.
I-5	Component IDs are registered atomically; no component may receive two different IDs.
I-6	ECS structural mutations during iteration go through `CommandBuffer`.
I-7	Job handles must check version equality before reporting completion.
I-8	`.caf` files are always verified with both CRC32 checks before use.
I-9	Near-plane clipping is applied to every projected line segment in Mode3D.
I-10	Navigation widget axes are computed from live `camYaw`/`camPitch` values, never hardcoded.

14.2 Planned Systems

The following systems are specified here to constrain future implementation choices, even though they are not yet implemented.

14.2.1 Mesh Rendering (Phase 5)

Mesh assets store interleaved vertex data (position, normal, UV, tangent) aligned to 32 bytes for AVX2 SIMD. The RHI layer (`src/rhi/`) wraps SDL3-GPU command buffers; mesh rendering will use indexed draws with a single persistent vertex buffer per mesh, uploaded once at asset load time.

14.2.2 Scripting (CppScript)

The scripting system (`src/script/`) uses a hot-reloadable C++ approach rather than an interpreted language. High-level logic is implemented by inheriting from the `CppScript` base class.

The `ScriptWatcher` monitors the filesystem for changes to script source files. Upon detection, it triggers an external compiler and dynamically reloads the resulting shared library (DLL/SO), patching the function pointers in live `ScriptComponent` instances. This provides the productivity of a scripting language with the full performance and type safety of C++.

14.2.3 Animation

Animation clips store keyframe data (time, value, tangent for Hermite interpolation) per channel (position, rotation, scale). The `AnimationSystem` evaluates clips via the Hermite spline formula and writes results into transform components.

Chapter 15

Render Hardware Interface

The Render Hardware Interface (RHI) serves as the foundational abstraction layer between the high-level rendering systems of the Caffeine Engine and the underlying graphics hardware. By utilizing the SDL3-GPU specification, the RHI provides a modern, stateless, and multithread-capable interface that unifies disparate graphics APIs such as Vulkan, Direct3D 12, and Metal into a single, cohesive programming model.

15.1 Design Rationale

The primary objective of the Caffeine RHI is to eliminate the overhead of traditional state-machine based APIs (e.g., OpenGL) while maintaining a level of abstraction that ensures cross-platform portability. The shift towards explicit graphics APIs necessitates a more disciplined approach to resource management and command submission.

SDL3-GPU Abstraction Rationale The choice of SDL3-GPU as the backend for the RHI is driven by three factors:

1. **Portability**: Unified access to Vulkan, D3D12, and Metal without maintaining separate backends.
2. **Modernity**: Unlike SDL_Render, SDL3-GPU provides low-level access to command buffers, pipelines, and descriptor sets.
3. **Stability**: It abstracts the volatile nature of raw Vulkan/D3D12 extensions into a stable API suitable for long-term engine development.

The RHI is designed around the concept of a *stateless command recording* phase followed by an *explicit submission* phase. This allows the engine to parallelize command recording across multiple CPU cores, effectively saturating the GPU command processor.

15.2 The Render Device

The `RenderDevice` class is the central orchestrator of the RHI. It manages the lifecycle of the physical and logical GPU devices, handles the creation of all hardware resources, and coordinates the triple-buffering logic required for smooth frame presentation.

15.2.1 Device Configuration and Initialization

Initialization of the `RenderDevice` requires a `RenderConfig` structure, which defines the initial state of the graphics subsystem.

```
struct RenderConfig {
    u32  width           = 1280;
    u32  height          = 720;
    bool vsync           = true;
    bool tripleBuffering = true;
    const char* windowTitle = "Caffeine Engine";
};
```

The configuration parameters serve the following purposes:

- **width** / **height**: Initial dimensions of the swapchain textures and backbuffer.
- **vsync**: Enables or disables Vertical Synchronization, mapping to the `SDL_GPU_PRESENTMODE_V` or `IMMEDIATE` modes.
- **tripleBuffering**: Determines the number of command buffers and resources maintained in flight to prevent CPU-GPU stalls.
- **windowTitle**: String identifier for the OS-level window handle.

15.2.2 Triple Buffering and Frame Management

The Caffeine Engine implements a triple-buffering strategy to maximize throughput. This is represented by the constant `MAX_FRAMES_IN_FLIGHT = 3`. Mathematically, the latency L of a frame can be expressed as:

$$L = \sum_{i=1}^{n} T_{CPU,i} + T_{GPU,i} \tag{15.1}$$

where n is the number of frames in flight. While triple buffering increases latency by one frame compared to double buffering, it ensures that the GPU never starves if the CPU takes slightly longer than $1/60$s to record commands.

15.3 Hardware Resources

Resources in the RHI are categorized into Textures, Buffers, and Shaders. All resources are created via the `RenderDevice` and are represented by opaque handles that wrap the underlying SDL3-GPU objects.

15.3.1 Textures

Textures represent multi-dimensional arrays of data, typically used for images, render targets, or depth-stencil buffers. The `Texture` struct maintains the handle and metadata:

```
struct Texture {
    SDL_GPUTexture* handle = nullptr;
    u32             width  = 0;
    u32             height = 0;
    TextureFormat   format = TextureFormat::Invalid;
};
```

Texture Formats

The `TextureFormat` enumeration defines the bit-layout of the texture data. The RHI supports a variety of formats optimized for different hardware paths:

- `R8G8B8A8_UNORM`: Standard 32-bit RGBA format with 8 bits per channel, normalized to $[0, 1]$.
- `B8G8R8A8_UNORM`: Blue-Green-Red-Alpha variant, often the native format for Windows-based swapchains.
- `R8_UNORM`: Single-channel 8-bit format, ideal for alpha masks or luminance.
- `R16_FLOAT`: 16-bit floating point, used for high-dynamic range (HDR) single-channel data.
- `R32G32B32A32_FLOAT`: Full 128-bit floating point format for high-precision compute or G-Buffer data.
- `D16_UNORM`, `D24_UNORM`, `D32_FLOAT`: Depth formats for the depth-stencil buffer.
- `SRGB` variants: Formats that implement automatic Gamma correction ($C_{linear} = C_{srgb}^{2.2}$).

Texture Usage Flags

Texture usage must be declared at creation time to allow the driver to optimize the memory layout (e.g., tiling).

```
enum class TextureUsage : u32 {
    Sampler      = SDL_GPU_TEXTUREUSAGE_SAMPLER,
    ColorTarget  = SDL_GPU_TEXTUREUSAGE_COLOR_TARGET,
    DepthStencil = SDL_GPU_TEXTUREUSAGE_DEPTH_STENCIL_TARGET,
    StorageRead  = SDL_GPU_TEXTUREUSAGE_GRAPHICS_STORAGE_READ,
    ComputeRead  = SDL_GPU_TEXTUREUSAGE_COMPUTE_STORAGE_READ,
    ComputeWrite = SDL_GPU_TEXTUREUSAGE_COMPUTE_STORAGE_WRITE,
};
```

15.3.2 Buffers

Buffers are linear regions of GPU memory used for vertex data, index data, and uniform constants.

```
struct Buffer {
    SDL_GPUBuffer* handle = nullptr;
    u64            size   = 0;
    BufferUsage    usage  = BufferUsage::Vertex;
};
```

The total memory allocated for a buffer S_{total} is often subject to alignment requirements:

$$S_{total} = \lceil S_{requested} / A \rceil \times A \tag{15.2}$$

where A is the hardware-specific alignment (typically 16 or 256 bytes for uniform buffers).

Buffer Usage

The `BufferUsage` enumeration specifies how the buffer will be accessed by the pipeline:

- `Vertex`: Contains per-vertex attributes.
- `Index`: Contains indices for indexed drawing.
- `Uniform`: Constant data accessible by shaders.
- `Storage`: Read-write data for compute or advanced graphics effects.
- `TransferSrc` / `TransferDst`: Used for staging data from CPU to GPU.

15.3.3 Shaders

Shaders are programmable stages of the graphics pipeline. The Caffeine Engine supports Vertex and Fragment (Pixel) shaders.

```
struct ShaderDesc {
    const u8*   code        = nullptr;
    usize       codeSize    = 0;
    const char* entryPoint  = "main";
    ShaderStage stage       = ShaderStage::Vertex;
    u32 numSamplers         = 0;
    u32 numStorageTextures  = 0;
    u32 numStorageBuffers   = 0;
    u32 numUniformBuffers   = 0;
};
```

The `ShaderDesc` requires explicit declaration of resource bindings (samplers, buffers) to facilitate the creation of pipeline layouts without expensive reflection at runtime.

15.4 Pipeline State

The `Pipeline` object encapsulates the entire state of the GPU for a draw call, including:

- Shader programs (Vertex and Fragment).
- Vertex input layout (attributes, strides).
- Primitive topology (Triangles, Lines).
- Rasterizer state (Culling, Fill mode).
- Multisample state.
- Depth-stencil state.
- Blend state for color attachments.

By baking this state into a single object, the RHI avoids the "state leakage" common in older APIs, where one draw call's state would unexpectedly affect the next.

15.5 Command Recording and Submission

The RHI uses a command-buffer based workflow to communicate with the GPU. This decoupled approach allows for efficient utilization of the hardware.

15.5.1 Command Buffers

A `CommandBuffer` is a recording of commands that the GPU will execute asynchronously.

Command Buffer Lifecycle The lifecycle of a command buffer in Caffeine follows a strict sequence:

1. **Acquisition**: A command buffer is acquired via `RenderDevice::beginFrame()`.
2. **Recording**: Commands are recorded (e.g., `bindPipeline`, `draw`).
3. **Submission**: The buffer is submitted via `RenderDevice::endFrame()`, at which point it is queued for GPU execution.
4. **Synchronization**: The engine ensures that the command buffer is not reused until the GPU has finished processing it.

15.5.2 Render Passes

All drawing operations must occur within a Render Pass. A Render Pass defines the targets (textures) being drawn to and the operations to perform at the start and end of the pass.

```
struct RenderPassDesc {
    f32  clearColor[4] = {0.0f, 0.0f, 0.0f, 1.0f};
    bool clearDepth    = false;
    f32  depthValue    = 1.0f;
};
```

When `beginRenderPass` is called, the RHI transitions the swapchain texture to the `COLOR_ATTACHMENT` state and clears it if specified by the `clearColor`.

15.5.3 Graphics Commands

Between `beginRenderPass` and `endRenderPass`, the `CommandBuffer` provides methods to bind resources and issue draws:

- `bindPipeline(Pipeline*)`: Sets the current graphics state.
- `bindVertexBuffer(Buffer*, slot)`: Binds a vertex buffer to a specific input slot.
- `bindIndexBuffer(Buffer*)`: Sets the buffer used for indexed drawing.
- `bindTexture(Texture*, slot)`: Binds a texture for sampling.

- `pushUniformData(stage, slot, data, size)`: Uploads small constants directly into the command stream.

The `DrawCommand` structure represents a high-level abstraction for a single draw call, often used by the `Renderer` to batch operations:

```
struct DrawCommand {
    Pipeline* pipeline      = nullptr;
    Buffer*   vertices      = nullptr;
    Buffer*   indices       = nullptr;
    Texture*  texture       = nullptr;
    u32       indexCount     = 0;
    u32       firstIndex    = 0;
    u32       instanceCount = 1;
    Mat4      transform     = Mat4::identity();
    Vec4      tint          = {1.0f, 1.0f, 1.0f, 1.0f};
    i32       sortKey       = 0;
};
```

The `sortKey` is utilized by the rendering front-end to sort draw calls by state (to minimize pipeline changes) or by depth (for opaque-to-transparent ordering).

15.6 Algorithmic Submission Flow

The submission flow of a frame in the Caffeine Engine can be described by the following algorithm:

1. **Frame Start**:
   - Wait for a free command buffer slot ($S_{frame} = m\_frameIndex \pmod 3$).
   - Request a `SDL_GPUCommandBuffer` from the device.
2. **Recording**:
   - Acquire the swapchain texture handle.
   - Begin the main render pass.
   - Loop through `DrawCommand` queue:
     (a) Bind pipeline if different from previous.
     (b) Bind vertex/index buffers.
     (c) Push `transform` and `tint` as uniform data.
     (d) Issue `drawIndexed`.

- End the main render pass.

3. **Frame End**:
   - Submit the command buffer.
   - Trigger the swapchain presentation.
   - Increment $m\_frameIndex$.

This algorithmic structure ensures that the GPU is kept busy with a continuous stream of work, minimizing idle time and maximizing the frame rate ($FPS = 1/T_{frame}$).

15.7 Conclusion

The Render Hardware Interface of the Caffeine Engine provides a robust and scalable foundation for modern real-time graphics. By abstracting the complexities of explicit APIs into a clean, handle-based system, it allows engine developers to focus on high-level rendering techniques while maintaining the performance characteristics of low-level hardware access.

Chapter 16

Two-Dimensional Rendering

The Two-Dimensional (2D) Rendering subsystem of the Caffeine Engine is architected to provide high-performance sprite and primitive rendering through an efficient batching and sorting pipeline. Designed to handle upwards of 30,000 sprites per frame while maintaining a low CPU footprint, the system leverages data-oriented design principles and modern graphics API features. This chapter explores the internal mechanisms of the batch renderer, the texture atlas management system, and the mathematical foundations of the orthographic camera.

16.1 Subsystem Architecture

The 2D rendering pipeline operates on the fundamental principle of minimizing GPU state changes, which are the primary bottleneck in modern graphics performance. In conventional rendering, each sprite might require a separate draw call, leading to significant overhead from driver-level context switching and uniform updates. Caffeine mitigates this through a deferred batching strategy.

The architecture is divided into three interconnected modules:

- **The Batch Renderer**: Responsible for collecting sprite primitives, sorting them by state and depth, and flushing them as large, indexed vertex buffers.
- **The Texture Atlas**: A spatial management system that packs multiple individual textures into a single hardware texture to reduce binding overhead.
- **The 2D Camera**: Provides the orthographic view-projection transformations and facilitates coordinate mapping between world-space and screen-space.

16.2 The BatchRenderer Pipeline

The `BatchRenderer` is the primary interface for 2D graphics submission. It abstracts the underlying Render Hardware Interface (RHI) and provides a high-level API for submitting transformed sprites.

16.2.1 Technical Specifications and Constraints

To maintain deterministic performance and prevent runtime memory fragmentation, the `BatchRenderer` defines strict upper bounds for its internal buffers. These constants are tuned for modern desktop hardware, balancing batch size with vertex data locality.

```
static constexpr u32 MAX_SPRITES_PER_BATCH = 32768;
static constexpr u32 MAX_VERTICES          = MAX_SPRITES_PER_BATCH
    * 4;
static constexpr u32 MAX_INDICES           = MAX_SPRITES_PER_BATCH
    * 6;
```

Listing 16.1: BatchRenderer Hardware Constraints

Each sprite is treated as a quad consisting of two triangles. For a batch of 32,768 sprites, the engine processes 131,072 vertices and 196,608 indices per draw call. This scale allows for complex 2D scenes with high particle counts or dense environments to be rendered in a single pass.

16.2.2 Vertex Format and Memory Layout

Efficiency in the vertex shader is directly linked to the memory layout of the vertex data. The `SpriteVertex` structure is optimized for 24-byte alignment, ensuring high throughput during GPU vertex fetching.

```
struct SpriteVertex {
    Vec3 position; // 12 bytes: x, y, z
    Vec2 texcoord; // 8 bytes: u, v
    u32  tint;     // 4 bytes: RGBA8 packed
};
```

Listing 16.2: SpriteVertex Memory Definition

Design Rationale: Packed Color Tints The tint field uses a single u32 to represent four 8-bit color channels. This choice reduces the vertex size from 36 bytes (using Vec4 floats for color) to 24 bytes, effectively reducing the memory bandwidth requirement by 33% without significant loss in color precision for 2D applications.

16.2.3 Submission and Primitives

During the application's update loop, sprites are submitted via the `submitSprite` method. Instead of immediate execution, the engine creates a `SpritePrimitive` which acts as a lightweight proxy for the final vertex data.

```
struct SpritePrimitive {
    Mat4 transform;
    Vec4 uv;
    u32  tint;
    u32  sortKey;
};
```

Listing 16.3: Internal SpritePrimitive Representation

16.2.4 State Sorting and Depth Encoding

Sorting is the most computationally expensive phase of the 2D pipeline but is essential for both correct transparency blending and performance. Caffeine uses a multi-faceted sort key to group similar rendering states together.

Sort Key Construction

The `sortKey` is a bit-packed 32-bit integer that encodes hierarchical sorting priorities. The construction logic ensures that coarse layers are prioritized, followed by texture binding, and finally fine-grained depth.

$$\text{Key} = ((\text{Layer} + 128) \& 0xFF) \ll 24 \mid (\text{TextureID} \& 0xFFF) \ll 12 \mid (\text{Depth}_{norm} \& 0xFFF) \tag{16.1}$$

The bit allocation is as follows:

- **Layer (8 bits)**: Supports 256 logical rendering layers. This allows for clear separation of background, midground, and UI elements.
- **Texture ID (12 bits)**: Groups up to 4,096 unique textures or atlas pages together.
- **Depth (12 bits)**: Provides 4,096 levels of Z-depth resolution within a single layer, enabling fine-grained interleaving of sprites.

Radix Sort Implementation

Because the sort key is an integer and the number of elements is large, the engine employs a Least Significant Digit (LSD) Radix Sort. Unlike comparison-based sorts like `std::sort` ($O(n \log n)$), Radix Sort operates in $O(n)$ time relative to the number of sprites.

The algorithm performs four passes, each processing 8 bits of the sort key. This approach is highly cache-friendly and avoids the branching overhead of traditional sorting algorithms.

Performance Advantage of Radix Sort For a typical load of 10,000 sprites, Radix Sort outperforms Quicksort by a factor of 3–5x. In the Caffeine Engine, the sorting pass is often the fastest part of the rendering frame, typically taking less than 0.2ms on a single thread.

16.2.5 Flushing and RHI Integration

The final stage of the rendering cycle is the `endFrame` call, where the sorted primitives are converted into hardware-ready buffers.

Quad Construction and Transformation

The engine iterates through the sorted `SpritePrimitive` list and generates four vertices per primitive. Each vertex is transformed from model-space (a unit quad centered at the origin) to world-space using the stored `Mat4` transform.

The UV coordinates are mapped according to the corners:

- Top-Left: `(uv.x, uv.w)`
- Top-Right: `(uv.z, uv.w)`
- Bottom-Right: `(uv.z, uv.y)`
- Bottom-Left: `(uv.x, uv.y)`

Transient Buffer Management

The generated vertex and index data are uploaded to the GPU. Caffeine utilizes transient buffers created via the `RenderDevice`. These buffers are often mapped directly into CPU memory space (using `MTLBuffer` on Metal or `vkMapMemory` on Vulkan) to minimize copying.

```
RHI::Buffer* vertexBuf = m_device->createBuffer(
    { static_cast<u64>(verts.size() * sizeof(SpriteVertex)),
       "BatchVertices" },
    RHI::BufferUsage::Vertex
);
cmd->bindVertexBuffer(vertexBuf);
cmd->drawIndexed(static_cast<u32>(indices.size()));
```

Listing 16.4: RHI Buffer Submission

16.3 Texture Atlas Management

Texture switches are one of the costliest operations in the graphics pipeline. The `TextureAtlas` class provides a mechanism to aggregate many small assets into a single large texture, thereby maximizing the efficiency of the `BatchRenderer`.

16.3.1 Shelf-Bin Packing Algorithm

The engine utilizes a Shelf-Bin Packing heuristic to organize sprites within the atlas. This algorithm is selected for its balance between packing density and computational speed.

Algorithm Formalization

The packing process involves three distinct phases: sorting, shelf allocation, and UV normalization.

1. **Sorting**: All sprites to be packed are sorted by their height in descending order. This ensures that the tallest sprite in any given "shelf" defines the shelf's vertical boundary, minimizing vertical gap wastage.
2. **Shelf Placement**: Primitives are placed horizontally along the current shelf. When the next sprite exceeds the atlas width, a new shelf is created at $Y_{current} + H_{shelf\_max}$.
3. **UV Generation**: Once placement is finalized, pixel coordinates are converted to normalized floating-point values $[0.0, 1.0]$.

Pseudocode and Logic

```
void TextureAtlas::pack() {
    std::sort(m_entries.begin(), m_entries.end(), [](const Entry&
       a, const Entry& b) {
        return a.srcH > b.srcH; // Descending height
    });

    u32 shelfX = 0, shelfY = 0, shelfH = 0;
    for (auto& e : m_entries) {
        if (shelfX + e.srcW > m_width) {
            shelfY += shelfH;
            shelfX = 0;
            shelfH = 0;
        }
        e.px = shelfX;
        e.py = shelfY;
        shelfX += e.srcW;
        shelfH = max(shelfH, e.srcH);
        // ... calculate normalized UVs ...
    }
}
```

Listing 16.5: Shelf-Packing Logic

16.3.2 Atlas Export and Runtime Generation

While the engine supports pre-baked atlases, the `TextureAtlas` class is fully capable of runtime generation. This is particularly useful for systems that generate textures procedurally or for loading assets from external sources at runtime.

The `exportImage()` method allows the engine to retrieve the raw pixel data from the packed atlas. This data can then be uploaded to a GPU texture object or saved to disk for debugging purposes.

```
struct PixelData {
    const u8* pixels   = nullptr;
    u32       width    = 0;
    u32       height   = 0;
    u32       byteSize = 0;
};
```

Listing 16.6: Atlas Pixel Export

Design Rationale: Atomic Packing The packing process is atomic. When new entries are added via the `add()` method, the packed flag is cleared. The atlas remains unpacked until the explicit `pack()` call is made. This prevents redundant re-packing operations when adding multiple sprites in sequence, ensuring that the heavy $O(n \log n)$ sort only occurs when necessary.

16.3.3 Utilization and Performance

The efficiency E of the atlas packing can be quantified as the ratio of used pixels to the total area of the atlas:

$$E = \frac{\sum_{i=1}^{n}(w_i \times h_i)}{W_{atlas} \times H_{atlas}} \tag{16.2}$$

For typical game assets, the Caffeine Engine's shelf-packer achieves an efficiency of 85–92%. While more complex algorithms like MaxRects provide higher density, the shelf-packer's $O(n \log n)$ complexity makes it suitable for runtime atlas generation during level loading.

16.4 The 2D Camera System

The `Camera2D` class provides the mathematical glue between the game world and the user's screen. It manages orthographic projection, view transformations, and procedural effects like screen shake.

16.4.1 Coordinate System and Viewport Mapping

Caffeine's 2D coordinate system uses a right-handed orientation where the positive X-axis points to the right and the positive Y-axis points upwards in world-space. However, standard screen coordinates (pixels) typically define the origin $(0, 0)$ at the top-left, with the positive Y-axis pointing downwards.

The `Camera2D` handles this discrepancy through its internal transformation pipeline. The viewport defined by the `Rect2D` structure determines the sub-region of the screen where the scene is rendered.

```
struct Rect2D {
    Vec2 position { 0.0f, 0.0f };
    Vec2 size     { 1280.0f, 720.0f };
};
```

Listing 16.7: Viewport Definition

16.4.2 Orthographic Projection Matrix

The projection matrix P maps the visible region of the world into Normalized Device Coordinates (NDC). In 2D, this is a linear mapping of the viewport dimensions, adjusted for the camera's zoom level.

Given a viewport with width W and height H, and a zoom factor z, the visible horizontal range is $[-\frac{W}{2z}, \frac{W}{2z}]$ and the vertical range is $[-\frac{H}{2z}, \frac{H}{2z}]$. The resulting matrix is:

$$P = \begin{bmatrix} \frac{2z}{W} & 0 & 0 & 0 \\ 0 & \frac{2z}{H} & 0 & 0 \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{16.3}$$

Where f and n represent the far and near planes, typically set to 1000 and -1000 to accommodate depth sorting.

16.4.3 View Transformation

The view matrix V represents the inverse of the camera's world-space transform. If the camera is at position $\mathbf{c} = (c_x, c_y)$ with a rotation θ, the view matrix is:

$$V = R_z(-\theta) \times T(-c_x, -c_y, 0) \tag{16.4}$$

This transformation ensures that world-space objects are shifted and rotated such that the camera's center appears at the origin of the screen.

16.4.4 Space Conversions

A critical requirement for gameplay systems (such as UI interaction and world-space targeting) is the ability to map coordinates between screen pixels and world units.

World-to-Screen Transformation

To convert a world position $\mathbf{p}_w$ to screen pixels $\mathbf{p}_s$:

1. Calculate NDC position: $\mathbf{p}_{ndc} = (P \times V) \times \mathbf{p}_w$
2. Scale and shift to screen space:

$$x_s = (x_{ndc} + 1) \cdot 0.5 \cdot W_{viewport} + x_{viewport} \tag{16.5}$$

$$y_s = (1 - y_{ndc}) \cdot 0.5 \cdot H_{viewport} + y_{viewport} \tag{16.6}$$

The inversion of the y component ($1 - y_{ndc}$) is necessary because screen coordinates usually define the origin $(0, 0)$ at the top-left, whereas NDC space defines it at the center with positive y pointing up.

Screen-to-World Transformation

The inverse process maps a pixel coordinate (x_s, y_s) back to the world. The engine performs this by first converting the screen coordinates to Normalized Device Coordinates (NDC), and then manually applying the inverse camera transformation.

Given a screen point $\mathbf{p}_s$, the NDC coordinates are:

$$x_{ndc} = \frac{x_s - x_{viewport}}{W_{viewport}} \cdot 2.0 - 1.0 \tag{16.7}$$

$$y_{ndc} = 1.0 - \frac{y_s - y_{viewport}}{H_{viewport}} \cdot 2.0 \tag{16.8}$$

The final world position $\mathbf{p}_w$ is then derived by considering the camera's zoom, rotation, and translation:

$$x_{cam} = x_{ndc} \cdot \frac{W_{viewport}}{2 \cdot zoom} \tag{16.9}$$

$$y_{cam} = y_{ndc} \cdot \frac{H_{viewport}}{2 \cdot zoom} \tag{16.10}$$

Applying the inverse rotation $R_z(\theta)$ and translation $T(c_x, c_y, 0)$:

$$x_w = x_{cam} \cos\theta - y_{cam} \sin\theta + c_x \tag{16.11}$$

$$y_w = x_{cam} \sin\theta + y_{cam} \cos\theta + c_y \tag{16.12}$$

Design Rationale: Manual Inverse vs Matrix Inverse While using Mat4::invert() on the View-Projection matrix is mathematically sound, the engine provides a manual implementation for screenToWorld. This avoids the numerical stability issues and computational cost associated with 4x4 matrix inversion for simple 2D transformations, resulting in cleaner and faster code for common gameplay tasks like mouse picking.

Dirty Flag Matrix Caching Matrix multiplication and inversion are computationally expensive. The Camera2D implements a "dirty flag" optimization. The View-Projection matrix is only recalculated when a camera property (position, rotation, zoom, or viewport) is modified. This reduces the per-frame overhead for static cameras to near zero.

16.4.5 Procedural Effects: Camera Shake

The camera system includes a integrated shake mechanism. When triggered, the effective position of the camera is perturbed by a random vector $\mathbf{s}$, which decays linearly over the duration of the shake.

$$\mathbf{c}_{effective} = \mathbf{c}_{base} + \mathbf{s}_{intensity} \cdot \left(\frac{t_{remaining}}{t_{duration}} \right) \cdot \text{rand}(-1, 1) \tag{16.13}$$

This effect is added before the view matrix construction, ensuring that the shake is correctly reflected in all rendered batches.

16.4.6 Performance Monitoring and Frame Statistics

To assist developers in optimizing their 2D scenes, the `BatchRenderer` maintains a set of high-level telemetry data. This information is updated at the end of each frame and can be retrieved via the `lastFrameStats()` method.

```
struct FrameStats {
    u32 totalSprites     = 0;
    u32 totalBatches     = 0;
    u32 drawCalls        = 0;
    u32 verticesUploaded = 0;
};
```

Listing 16.8: FrameStats telemetry structure

These metrics provide critical insight into the efficiency of the batching process. For instance, a high ratio of `drawCalls` to `totalSprites` indicates that the texture atlas is not being utilized effectively, or that state changes (such as layer switches) are forcing premature flushes.

Interpreting Telemetry In a perfectly optimized scene where all sprites share a single atlas and reside on the same layer, the drawCalls and totalBatches count should both be 1, regardless of the number of sprites (up to the hardware limit). Any deviation from this suggests an opportunity for better asset organization.

16.5 Summary

The Two-Dimensional Rendering subsystem of the Caffeine Engine represents a sophisticated blend of algorithmic efficiency and architectural clarity. By centralizing sprite submission into the `BatchRenderer`, the engine maximizes GPU utilization through state-sorting and batching. The addition of the `TextureAtlas` system and the mathematically rigorous `Camera2D` provides developers with a powerful toolset for creating high-performance, visually rich 2D experiences. In subsequent chapters, we will examine how this system integrates with the global lighting and post-processing pipelines.

Chapter 17

Three-Dimensional Rendering

The three-dimensional rendering subsystem of the Caffeine Engine provides a robust framework for managing spatial complexity and visual fidelity. This chapter covers the architectural components responsible for view management, visibility determination, spatial partitioning, and light representation.

17.1 The Camera3D Subsystem

The `Camera3D` class serves as the primary interface between the virtual world and the rendering viewport. It encapsulates the mathematical transformations necessary to project three-dimensional world coordinates into two-dimensional screen space while supporting multiple interaction paradigms.

17.1.1 Operational Modes

Caffeine supports three distinct camera behaviors, each tailored for specific gameplay or tool-based scenarios. These modes dictate how the camera responds to input and its relationship with other entities in the world.

First-Person Perspective (FPS)

The FPS mode implements traditional mouse-look and keyboard-based navigation. It maintains internal pitch and yaw angles, where pitch is clamped to ± 89 degrees to avoid gimbal lock and orientation inversion. Movement is calculated by projecting input vectors onto the camera's local horizontal plane, ensuring that vertical tilt does not affect movement speed when the player looks up or down.

Orbital Navigation

Designed for editors and strategy games, this mode rotates around a fixed `orbitTarget`. It uses azimuth and elevation parameters combined with an `orbitDistance` to position the camera on a spherical shell. The position in Cartesian coordinates is calculated as:

$$x = r \cdot \cos(\phi) \cdot \sin(\theta), \quad y = r \cdot \sin(\phi), \quad z = r \cdot \cos(\phi) \cdot \cos(\theta) \tag{17.1}$$

where r is the distance, ϕ is the elevation, and θ is the azimuth.

Entity Following

The Follow mode attaches the camera to a target `ECS::Entity`. It employs linear interpolation (lerp) for smooth movement, allowing the camera to lag slightly behind the target for a more fluid feel. The `followSmoothing` factor determines the responsiveness of the camera to the target's position changes.

17.1.2 Mathematical Derivation of the Basis Vectors

The orientation of the camera is represented internally by a unit quaternion q. To calculate the local basis vectors (Forward, Right, and Up) in world space, the engine rotates the standard axis vectors:

$$\vec{f} = q \cdot \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \cdot q^{-1}, \quad \vec{r} = q \cdot \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \cdot q^{-1}, \quad \vec{u} = q \cdot \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \cdot q^{-1} \tag{17.2}$$

These vectors form an orthonormal basis. The `forward()` vector points into the screen, `right()` points to the right side of the viewport, and `up()` aligns with the vertical axis relative to the camera's tilt.

17.1.3 Perspective Projection Matrix

The engine uses a standard right-handed perspective projection matrix. Given a vertical field of view θ (fovY) in radians, an aspect ratio a, and distances to the near (n) and far (f) clipping planes, the matrix P is constructed as follows.

First, the focal length g is defined:

$$g = \frac{1}{\tan(\theta/2)} \tag{17.3}$$

The projection matrix P is then:

$$P = \begin{pmatrix} g/a & 0 & 0 & 0 \\ 0 & g & 0 & 0 \\ 0 & 0 & -(f+n)/(f-n) & -2fn/(f-n) \\ 0 & 0 & -1 & 0 \end{pmatrix} \tag{17.4}$$

This matrix maps the viewing frustum to a normalized device coordinate (NDC) cube ranging from -1 to 1 on all axes.

Design Rationale: Matrix Convention Caffeine employs column-major matrices to maintain compatibility with OpenGL-style APIs. However, the internal math library provides helper functions to abstract away the memory layout, ensuring that developers can focus on the geometric logic rather than memory offsets.

17.2 Frustum Culling and Visibility

To maintain high performance in complex scenes, the engine must quickly discard objects that are not visible to the camera. This is achieved through frustum culling.

17.2.1 Frustum Construction

The `Frustum` struct represents the viewing volume as six planes. Each plane is defined by the equation $n \cdot x + d = 0$, where n is the normal vector pointing towards the outside of the frustum.

The `fromCamera` method builds these planes using the camera's position, forward vector, and projection parameters. For example, the near plane is defined at distance $nearZ$ from the eye position E:

$$n_{near} = -\vec{f}, \quad d_{near} = -n_{near} \cdot (E + \vec{f} \cdot nearZ) \tag{17.5}$$

Side planes are derived by rotating the view-space normals into world space. If $tanH = \tan(\theta/2) \cdot a$, the left plane normal in view space is $(1, 0, -tanH)$. This normal is transformed using the camera's rotation matrix.

17.2.2 Plane-AABB Intersection

To test if an Axis-Aligned Bounding Box (AABB) is inside the frustum, the engine performs a plane-box intersection test. For each of the six planes, it finds the "p-vertex" (the corner of the AABB furthest in the direction of the plane's normal).

If p is the p-vertex of the AABB for plane P_i, and $dist(p, P_i) > 0$, the box is entirely outside the frustum and can be culled. The signed distance for a point (x, y, z) to a plane (A, B, C, D) is:

$$d = A \cdot x + B \cdot y + C \cdot z + D \tag{17.6}$$

Sphere and Point Visibility

In addition to AABB tests, the engine supports sphere and point visibility checks. A point is visible if it lies on the "inside" side of all six frustum planes. For a sphere with center C and radius R, the test checks if the distance from C to any plane exceeds R on the "outside" side:

$$isVisible = \forall i \in \{1..6\} : (n_i \cdot C + d_i) \leq R \tag{17.7}$$

This allows for fast culling of particle systems or simplified trigger volumes.

17.3 Octree Spatial Partitioning

The `Octree` class provides a hierarchical organization of 3D space. It allows for efficient spatial queries, such as finding all entities within a frustum or intersecting a ray.

17.3.1 Recursive Subdivision

An Octree begins as a single root node covering the entire scene boundaries. When the number of entities in a leaf node exceeds `maxEntitiesPerNode`, the node is subdivided into eight children.

The split point is always the center of the current node's AABB. Each child represents one octant of the parent's volume. The subdivision continues until the `maxDepth` is reached.

```
1 void subdivide() {
2     Vec3 c = bounds.center();
3     for (int i = 0; i < 8; ++i) {
```

```
        auto ch = std::make_unique<Node>();
        ch->bounds.min = {
            (i & 1) ? c.x : bounds.min.x,
            (i & 2) ? c.y : bounds.min.y,
            (i & 4) ? c.z : bounds.min.z
        };
        ch->bounds.max = {
            (i & 1) ? bounds.max.x : c.x,
            (i & 2) ? bounds.max.y : c.y,
            (i & 4) ? bounds.max.z : c.z
        };
        ch->depth  = depth + 1;
        ch->isLeaf = true;
        children[i] = std::move(ch);
    }
    isLeaf = false;
}
```

Listing 17.1: Octree Node Subdivision

17.3.2 Ray-AABB Intersection (Slab Method)

Raycasting through the Octree uses the slab method for efficient AABB intersection. For a ray with origin O and direction D, and a box defined by $[min, max]$, the intersection intervals for each axis i are:

$$t_{1,i} = \frac{min_i - O_i}{D_i}, \quad t_{2,i} = \frac{max_i - O_i}{D_i} \tag{17.8}$$

The entry and exit times are:

$$t_{min} = \max(\min(t_{1,x}, t_{2,x}), \min(t_{1,y}, t_{2,y}), \min(t_{1,z}, t_{2,z})) \tag{17.9}$$

$$t_{max} = \min(\max(t_{1,x}, t_{2,x}), \max(t_{1,y}, t_{2,y}), \max(t_{1,z}, t_{2,z})) \tag{17.10}$$

An intersection occurs if $t_{max} \geq t_{min}$ and $t_{max} \geq 0$.

17.3.3 Query Architectures

The Octree supports multiple query types to help different engine subsystems, such as physics, AI, and rendering.

Frustum Query Traversal

Querying the Octree with a frustum is a recursive process. At each node, the engine tests the node's AABB against the frustum. If the AABB is completely outside, the entire branch is discarded. If the node is a leaf, every entity's AABB is tested against the frustum. Visible entities are added to the result buffer.

Radius and Sphere Queries

For proximity-based logic, such as explosion damage or local light influence, the Octree provides radius queries. This search identifies all entities whose AABB center lies within a specified distance from a point. The engine optimizes this by first checking if the query sphere intersects the node's AABB using a clamped distance squared check:

$$distSq = \sum_{i \in \{x,y,z\}} (\max(0, min_i - C_i) + \max(0, C_i - max_i))^2 \tag{17.11}$$

If $distSq \leq R^2$, the traversal continues into the node.

17.4 Lighting Components

Caffeine represents light sources as ECS components. The rendering engine traverses these components to build the lighting data sent to the GPU.

17.4.1 Base Light Properties

Every light source shares a common `LightComponent` which defines the color and intensity. The color is stored as a `Vec4` (RGBA), while intensity is a scalar multiplier applied during the lighting calculation.

```
struct LightComponent {
    Vec4 color = Vec4(1.0f, 1.0f, 1.0f, 1.0f);
    f32 intensity = 1.0f;
};
```

17.4.2 Specific Light Types

The engine supports three primary light types, each with unique geometric properties and rendering requirements.

Directional Lighting

Directional lights simulate distant sources like the sun. The rays are parallel, so the light position is ignored. The direction vector $\vec{L}$ is the only geometric parameter. The lighting shader uses this vector directly in the Lambertian diffuse calculation:

$$I = \max(0, \vec{n} \cdot -\vec{L}) \tag{17.12}$$

This component includes a `shadowDistance` property to limit the range of shadow mapping, optimizing depth buffer usage.

Point Lighting

Point lights emit light omnidirectionally from a position P. The intensity diminishes over distance d. Caffeine uses a radius-based attenuation model where the light contribution drops to zero at distance R. The attenuation factor A is typically calculated as:

$$A = \max(0, 1 - (d/R)^2) \tag{17.13}$$

This ensures a smooth transition at the edge of the light's influence.

Spot Lighting

Spot lights emit light in a cone defined by a direction $\vec{L}$ and an `angle` ϕ. The angular attenuation S is determined by the cosine of the angle between the light direction and the vector to the fragment $\vec{V}$:

$$S = \text{smoothstep}(\cos(\phi_{outer}), \cos(\phi_{inner}), \vec{L} \cdot \vec{V}) \tag{17.14}$$

This creates a soft falloff at the edges of the cone, preventing harsh geometric artifacts.

17.5 Mesh and Rendering State

Entities that appear in the 3D world must possess components that describe their geometry and material properties.

17.5.1 MeshRendererComponent

This component links an entity to a static mesh asset. It contains the paths to the mesh data and the material file. It also holds flags for shadow behavior.

```
struct MeshRendererComponent {
    std::string meshPath;
    std::string materialPath;
    bool castShadows = true;
    bool receiveShadows = true;
};
```

17.5.2 MeshFilterComponent

For procedural or primitive-based geometry, the `MeshFilterComponent` specifies the type of primitive to generate (Cube, Sphere, Capsule, etc.) or a path to a custom mesh. This allows for rapid prototyping without requiring external asset files for basic shapes.

Implementation Detail: Skinned Meshes The `SkinnedMeshRendererComponent` extends the basic mesh concept by adding a `skeletonPath`. This triggers a different rendering path that uses vertex skinning on the GPU, allowing for character animations where the mesh deforms based on an underlying bone hierarchy.

Chapter 18

Event System

The Caffeine Engine employs a decoupled communication architecture through its `EventBus` subsystem. This component facilitates many-to-many communication without requiring explicit dependencies between producers and consumers. By utilizing a zero-RTTI type identification mechanism, the system maintains high performance while providing a flexible, type-safe interface for event subscription and dispatch.

18.1 Type Identification Mechanism

A core challenge in a generic event system is identifying event types at runtime without the overhead of C++ Run-Time Type Information (RTTI). Caffeine solves this using a static sentinel address technique.

Design Rationale: Zero-Cost Type IDs Standard RTTI (`typeid`) can introduce significant binary size overhead and performance penalties. Instead, Caffeine utilizes the unique memory address of a static variable within a template function to generate unique identifiers at compile-time for each distinct type.

The implementation resides in the `Detail::eventTypeId<T>` function:

```
template<typename T>
u32 eventTypeId() {
    static char s_sentinel = 0;
    return
       static_cast<u32>(reinterpret_cast<uintptr_t>(&s_sentinel));
}
```

18.1.1 How it Works

When the compiler instantiates a template function for a specific type `T`, it generates a unique instance of that function. Each instance contains its own static storage for the `s_sentinel` variable. According to the C++ standard, these static variables are guaranteed to have unique addresses across the entire program (per translation unit, effectively merged by the linker).

By casting the address of this sentinel to a `u32`, the engine obtains a unique integer ID for every event type. This process happens entirely without string comparisons or complex hash calculations, resulting in a true $O(1)$ type lookup.

18.2 Event Subscription

Consumers register their interest in specific events via the `subscribe<T>` method. This method accepts a callback function and returns a `ListenerHandle`.

```
template<typename T>
ListenerHandle subscribe(std::function<void(const T&)> callback) {
    const u32 typeId = Detail::eventTypeId<T>();
    const ListenerHandle handle = m_nextHandle++;

    Detail::ListenerRecord record{
        handle,
        [cb = std::move(callback)](const void* data) {
            cb(*static_cast<const T*>(data));
        }
    };

    m_listeners[typeId].push_back(std::move(record));
    return handle;
}
```

18.2.1 Listener Records and Callbacks

Internally, the `EventBus` stores subscribers in a `std::unordered_map` where keys are the type IDs and values are vectors of `ListenerRecord` objects. Since the bus stores disparate event types, it uses type erasure via `std::function<void(const void*)>` to store the callbacks. When an event is published, the bus casts the generic pointer back to the specific event type before invoking the user's callback.

18.2.2 The ListenerHandle Lifecycle

The `ListenerHandle` is an opaque unsigned integer used to identify a specific subscription. It's essential for unsubscription:

```
void unsubscribe(ListenerHandle handle);
```

When `unsubscribe` is called, the `EventBus` searches its listener maps and removes the record associated with that handle. It is the caller's responsibility to manage the lifecycle of this handle. If a listener object is destroyed without unsubscribing, the `EventBus` may attempt to call a dangling pointer or an invalid functional object, leading to undefined behavior.

18.3 Event Dispatching

The `EventBus` supports two modes of dispatch: immediate synchronous dispatch and deferred queue-based dispatch.

18.3.1 Immediate Dispatch

The `publish<T>` method triggers an immediate broadcast of the event to all current subscribers.

```
template<typename T>
void publish(const T& event) {
    const u32 typeId = Detail::eventTypeId<T>();
    auto it = m_listeners.find(typeId);
    if (it == m_listeners.end()) return;

    std::vector<Detail::ListenerRecord> snapshot = it->second;
    for (const auto& record : snapshot) {
        record.callback(static_cast<const void*>(&event));
    }
}
```

Note that the system takes a snapshot of the listeners before dispatching. This prevents issues if a listener attempts to unsubscribe or subscribe to new events during the dispatch loop, which would otherwise invalidate the iterators of the internal vector.

18.3.2 Deferred Dispatch

In many engine scenarios, such as within a physics update or a rendering loop, immediate event handling might be undesirable due to performance or state consistency reasons. For these cases, `publishDeferred<T>` pushes events into a queue.

```
template<typename T>
void publishDeferred(T event) {
    const u32 typeId = Detail::eventTypeId<T>();
    std::lock_guard<std::mutex> lock(m_queueMutex);
    m_queue.push_back(
        std::make_unique<Detail::EventWrapper<T>>(typeId,
           std::move(event)));
}
```

Events in this queue are not processed until `dispatch()` (or `flush()`) is explicitly called.

18.3.3 Queue Processing

The `dispatch()` method iterates through the accumulated events in the deferred queue and broadcasts them to their respective subscribers. Each event is wrapped in an `EventWrapper` which inherits from a common `IEventWrapper` interface, allowing the queue to store events of different types polymorphically.

18.4 Thread Safety and Concurrency

The `EventBus` is designed to be partially thread-safe to support the engine's multi-threaded nature.

18.4.1 Mutex Usage

The deferred event queue is protected by a `std::mutex` (`m_queueMutex`). This allows multiple threads to safely call `publishDeferred` simultaneously. The use of `std::lock_guard` ensures that the mutex is always released, even if an exception occurs during the push operation.

18.4.2 Synchronous Limitations

The `m_listeners` map and `subscribe`/`unsubscribe` operations are not currently protected by internal mutexes. This is a deliberate design choice to avoid the significant overhead of locking on every event lookup. As a result, subscriptions and unsubscriptions should typically happen on a single primary thread (usually the main engine thread or a dedicated event thread) or be externally synchronized by the caller.

18.5 Best Practices

To ensure optimal use of the event system, developers should follow these guidelines:

1. **Small Event Objects**: Events are often copied (especially in deferred dispatch). Keep event structures small or use `std::move` where applicable.
2. **Lifecycle Management**: Always unsubscribe when a listener is destroyed. Using a scoped guard or a RAII wrapper for the `ListenerHandle` is recommended.
3. **Prefer Deferred for Heavy Logic**: If an event triggers complex logic, use `publishDeferred` to avoid blocking the critical path of the producer.
4. **Avoid Deep Nesting**: Publishing an event within a callback of another event can lead to complex call stacks and potential deadlocks if not handled carefully.

Chapter 19

Input Management

The Caffeine Engine implements an action-based input system designed to decouple physical hardware inputs from logical game logic. This abstraction allows developers to define semantic actions such as "Jump" or "Interact" without hardcoding specific keys or buttons. By providing a layer of indirection, the system helps with rebindable controls, multi-device support, and simplified input handling across different platforms.

19.1 Logical Abstractions

At the core of the input subsystem are two primary abstractions: Actions and Axes. Actions represent discrete events, while Axes represent continuous ranges of motion or values.

19.1.1 Action Enumeration

The `Action` enum defines the semantic operations available within the engine. Each action corresponds to a specific gameplay intent rather than a physical key.

```
enum class Action : u8 {
    MoveUp = 0,
    MoveDown,
    MoveLeft,
    MoveRight,
    Jump,
    Attack,
    Interact,
    Pause,
    Count
};
```

- **MoveUp, MoveDown, MoveLeft, MoveRight**: Standard directional movements often mapped to WASD or D-pad.
- **Jump**: Triggers a character jump or vertical traversal.
- **Attack**: Primary offensive action.
- **Interact**: Contextual interaction with objects in the world.
- **Pause**: Accesses the game menu or halts simulation.

19.1.2 Axis Enumeration

Axes handle directional input that requires a scalar value, such as analog stick movement or mouse motion.

```
enum class Axis : u8 {
    MoveX = 0,
    MoveY,
    LookX,
    LookY,
    Count
};
```

The `MoveX` and `MoveY` axes typically handle player movement, while `LookX` and `LookY` are dedicated to camera control.

19.2 Hardware Mappings

The engine translates raw hardware signals into the logical abstractions mentioned above. To maintain portability, the system uses constants compatible with SDL3 scan codes but remains independent of the SDL3 headers during compilation.

19.2.1 Keyboard and Mouse Codes

The `Key` enum provides a comprehensive list of keyboard scan codes. These values follow the SDL3 scan code specification, ensuring consistent behavior across different keyboard layouts.

```
enum class Key : u16 {
    Unknown = 0,
    A = 4, B, C, D, E, F, G, H, I, J, K, L, M,
    N, O, P, Q, R, S, T, U, V, W, X, Y, Z,
```

```
    Num1 = 30, Num2, Num3, Num4, Num5, Num6, Num7, Num8, Num9,
        Num0,
    Return = 40, Escape, Backspace, Tab, Space,
    Up = 82, Down, Left = 80, Right,
    LShift = 225, RShift, LCtrl = 224, RCtrl,
    LAlt = 226, RAlt,
    KeyCount = 512
};
```

Mouse input is handled through the `MouseButton` enum, supporting standard left, middle, and right clicks, along with two additional auxiliary buttons.

```
enum class MouseButton : u8 {
    Left = 1,
    Middle,
    Right,
    X1,
    X2,
    Count
};
```

19.2.2 Gamepad Support

Caffeine provides native support for modern gamepads via the `GamepadButton` and `GamepadAxis` enumerations. The button layout follows the standard Xbox/PlayStation controller configurations.

```
enum class GamepadButton : u8 {
    A = 0, B, X, Y,
    LeftBumper, RightBumper,
    Back, Start, Guide,
    LeftStick, RightStick,
    DPadUp, DPadDown, DPadLeft, DPadRight,
    Count
};

enum class GamepadAxis : u8 {
    LeftX = 0,
    LeftY,
    RightX,
    RightY,
    TriggerLeft,
    TriggerRight,
```

```
    Count
};
```

Gamepad Deadzones While the `InputManager` tracks raw axis values, it's common practice to apply deadzone filtering at the gameplay logic level. The `GamepadAxis` values are stored as floating point numbers ranging from -1.0 to 1.0, or 0.0 to 1.0 for triggers.

19.3 Binding Mechanism

The binding system connects physical inputs to logical actions. A single action can be bound to multiple physical inputs (up to 4 by default), allowing for "primary" and "secondary" controls, such as mapping `Jump` to both the Space bar and the Gamepad A button.

19.3.1 The Binding Structure

The `Binding` struct uses a tagged union to store the type of input and the corresponding code. This compact representation makes it easy to pass bindings as parameters and store them in arrays.

```
struct Binding {
    BindingType type;
    union {
        Key             key;
        MouseButton     mouseButton;
        GamepadButton   gamepadButton;
        GamepadAxis     gamepadAxis;
    };
    // Helper static methods: fromKey, fromMouseButton, etc.
};
```

19.3.2 Registering Bindings

To bind a key to an action, the `bind` method is used. For axes, the `bindAxis` method allows mapping two separate inputs to represent the negative and positive directions of a single logical axis.

```
// Example: Binding MoveLeft to 'A' and MoveRight to 'D' for a
   logical X axis
inputManager.bindAxis(Axis::MoveX,
                      Binding::fromKey(Key::A),
                      Binding::fromKey(Key::D));
```

The engine also provides methods to clear bindings for specific actions or reset the entire system to its default configuration via `resetToDefaults()`.

19.4 State Management and Polling

The `InputManager` maintains the current and previous state of all inputs to provide accurate frame based queries. This allows the system to distinguish between a key that's currently held down and a key that was pressed exactly during the current frame.

19.4.1 Action and Axis States

Queries for logical actions return an `ActionState` object, which contains three boolean flags.

```
struct ActionState {
    bool pressed      = false;
    bool justPressed  = false;
    bool justReleased = false;
};
```

- **pressed**: True if the action is currently active.
- **justPressed**: True only during the first frame the action becomes active.
- **justReleased**: True only during the frame the action stops being active.

Continuous inputs return an `AxisState`, providing the current scalar value and the delta since the last frame.

19.4.2 The Frame Lifecycle

Proper state tracking requires consistent updates at the start and end of every frame. The `beginFrame()` method caches the current state into the "previous state" buffer, while `endFrame()` prepares the manager for the next cycle.

During the `beginFrame()` method, the following operations occur:

1. Copy key state to previous key state.
2. Copy mouse state to previous mouse state.
3. Store current mouse position as previous mouse position.
4. Clear temporary injection buffers.

This double buffering ensures that logic running during the frame can query `isActionJustPresse` reliably, regardless of when the actual hardware event was received.

This double buffering ensures that logic running during the frame can query `isActionJustPresse` reliably, regardless of when the actual hardware event was received.

19.5 Event Injection and Callbacks

To keep the input system decoupled from the windowing library (SDL3), the `InputManager` doesn't poll hardware directly. Instead, it exposes an injection API. The platform layer or windowing system captures raw events and "injects" them into the manager.

```
void injectKeyDown(Key key);
void injectMouseButtonDown(MouseButton button);
void injectMouseMove(f32 x, f32 y);
```

This architecture makes the engine highly testable, as inputs can be simulated in unit tests without a physical keyboard or mouse.

19.5.1 The Callback Interface

While polling is the preferred method for most gameplay logic, some systems require event driven notifications. The engine provides two ways to listen for input events: a functional interface using `std::function` and a virtual interface through the `IInputCallbacks` class.

```
class IInputCallbacks {
public:
    virtual ~IInputCallbacks() = default;
    virtual void onActionPressed(Action action) = 0;
    virtual void onActionReleased(Action action) = 0;
};
```

By implementing this interface and registering it with `setCallbackHandler()`, a system can receive immediate notifications when an action state changes, which is useful for UI systems or event based triggers.

Chapter 20

Debugging and Profiling

20.1 Logging System

The Caffeine Engine incorporates a centralized logging system to provide detailed insights into the runtime behavior of the engine and game code. The `LogSystem` class serves as the primary interface for message output, offering features like level-based filtering, category-based isolation, and extensible message sinks.

Design Rationale: Fixed Buffer Logging To maintain performance during intense debugging sessions, the logging system uses a fixed-length message buffer of 2048 characters. This avoids heap allocations during the formatting process, ensuring that logging calls remain relatively lightweight and don't introduce significant jitter in the frame rate.

20.1.1 System Architecture

The logging system is implemented as a singleton, ensuring a single point of truth for all diagnostic messages. It employs a thread-safe design using a mutex to synchronize access to internal state and message sinks, allowing multiple threads to log concurrently without corrupting the output.

```
class LogSystem {
public:
    static LogSystem& instance();

    void log(LogLevel level, const char* category, const char*
       fmt, ...);
    void vlog(LogLevel level, const char* category, const char*
       fmt, va_list args);

    void setLevel(LogLevel minLevel);
```

```
    LogLevel getLevel() const;

    void setCategoryEnabled(const char* category, bool enabled);
    bool isCategoryEnabled(const char* category) const;

    using SinkFn = std::function<void(LogLevel, const char*, const
        char*)>;
    void addSink(SinkFn sink);
    void clearSinks();

private:
    static constexpr usize MAX_CATEGORIES = 64;
    static constexpr usize MAX_SINKS = 16;
    static constexpr usize MAX_MESSAGE_LENGTH = 2048;

    LogLevel m_minLevel = LogLevel::Trace;
    CategoryEntry m_categories[MAX_CATEGORIES]{};
    SinkFn m_sinks[MAX_SINKS]{};
    mutable std::mutex m_mutex;
};
```

Listing 20.1: LogSystem Class Definition

20.1.2 Logging Levels

The engine defines five distinct severity levels to categorize messages. This classification allows developers to filter out less critical information during normal operation while retaining the ability to capture granular details when troubleshooting specific issues.

- **Trace**: Highly detailed information, typically only useful during the development of specific features.
- **Info**: General operational messages that highlight significant engine milestones or state changes.
- **Warn**: Indications of potential problems or unusual conditions that don't prevent the engine from running.
- **Error**: Significant failures that impact specific functionality but might allow the application to continue.
- **Fatal**: Critical errors that result in immediate application termination or unrecoverable state.

20.1.3 Categories and Filtering

Messages are further organized into categories, which act as namespaces for log entries. This system prevents the log output from becoming cluttered by allowing developers to enable or disable specific engine subsystems individually. For example, one might enable logging for the `"Memory"` category while keeping `"Renderer"` silent.

The system supports a maximum of 64 unique categories. Each category is tracked using a name buffer of 64 characters and a boolean flag indicating its enabled state. When a log call is made with a new category name, the system automatically registers it if space is available.

20.1.4 Message Sinks

The `LogSystem` doesn't dictate where messages are displayed. Instead, it uses a sink-based architecture. Developers can register up to 16 callback functions of type `SinkFn`. When a message passes the level and category filters, it's dispatched to all registered sinks.

This flexibility allows logs to be simultaneously routed to the console, written to a file, displayed in an in-game overlay, or sent to a remote debugging server. Sinks receive the raw log level, the category name, and the formatted message string.

20.2 Performance Profiling

Optimizing a game engine requires precise measurements of how long specific operations take. The `Profiler` subsystem provides a low-overhead solution for gathering timing statistics across the codebase. It tracks execution times for named scopes and aggregates data to identify performance bottlenecks.

20.2.1 RAII-based Profiling

The primary way to profile code is through the `ProfileScope` class. This RAII (Resource Acquisition Is Initialization) guard records the start time in its constructor and calculates the elapsed duration in its destructor. By wrapping a block of code with this guard, developers can easily measure its performance without manual timing calls.

```
class ProfileScope {
public:
    explicit ProfileScope(const char* name) : m_name(name) {
        Profiler::instance().beginScope(name);
    }
    ~ProfileScope() {
```

```
        Profiler::instance().endScope(m_name);
    }
private:
    const char* m_name;
};

#define CF_PROFILE_SCOPE(name) \
    Caffeine::Debug::ProfileScope _cfProfileScope_##__LINE__(name)
```

Listing 20.2: Profiling Macros and RAII Guard

Using a macro for profiling scopes ensures that each instance has a unique variable name based on the line number, preventing name collisions within the same scope.

20.2.2 Data Collection and Statistics

The `Profiler` singleton maintains an array of up to 256 scopes. For each scope, the system tracks several key metrics to provide a comprehensive view of performance over time.

```
struct ScopeStats {
    const char* name = nullptr;
    u64  callCount = 0;
    f64  totalMs   = 0.0;
    f64  avgMs     = 0.0;
    f64  minMs     = 1e18;
    f64  maxMs     = 0.0;
};
```

Listing 20.3: Scope Statistics Structure

When a scope ends, the profiler updates the following values:

- **Call Count**: The total number of times the scope was entered since the last reset.
- **Total Time**: The cumulative time spent inside the scope, measured in milliseconds.
- **Min/Max Time**: The shortest and longest recorded durations for a single execution of the scope.

Average time is calculated during reporting by dividing the total time by the call count. This data aggregation happens in-place within the `InternalScopeData` structure, which also includes the active timer for the current execution.

20.2.3 Timing Precision

Timing in the Caffeine Engine relies on the `Core::Timer` class. This timer uses a high-resolution clock provided by the underlying platform, typically through SDL3's performance counter.

The system operates with a resolution of one million ticks per second, translating to microsecond precision. When calculating durations, the tick difference between the start and end points is divided by 1,000,000 to convert the value into seconds. The profiler then converts these seconds into milliseconds for more readable statistics.

Design Rationale: Fixed Scope Storage The profiler uses a pre-allocated array of 256 InternalScopeData entries. While this limits the number of unique scopes that can be tracked simultaneously, it guarantees that entering or exiting a profiled region never triggers a memory allocation. This is vital for profiling tight loops or high-frequency functions where the overhead of a hash map or dynamic vector would skew the results.

20.2.4 Reporting and Resetting

The profiler provides a `report` method that populates a vector with the current statistics for all active scopes. This separation of data collection and reporting allows the engine to gather data during the main loop and display it at a lower frequency, such as once per second or upon user request.

The `reset()` method clears all accumulated statistics and resets the scope count to zero. This is usually called at the start of a new profiling session or when switching game states to ensure that the data reflects the current workload accurately.

20.2.5 Practical Usage

To profile a function, a developer simply needs to add the `CF_PROFILE_SCOPE` macro at the beginning of the function body. The engine handles the rest, including registering the scope name and updating the statistics on every call.

```
1 void Renderer::drawFrame() {
2     CF_PROFILE_SCOPE("Renderer::drawFrame");
3
4     if (!m_initialized) {
5         CF_ERROR("Renderer", "Attempting to draw with
              uninitialized renderer");
6         return;
7     }
8
9     // Drawing logic...
```

```
    CF_TRACE("Renderer", "Frame submitted to GPU");
}
```

Listing 20.4: Example Usage of Profiling and Logging

By combining granular logging with high-precision profiling, the Caffeine Engine provides a robust framework for diagnosing issues and verifying performance targets during development.

Chapter 21

Scene Management

The Scene Management subsystem in the Caffeine Engine provides a robust framework for handling game states, level transitions, and spatial hierarchies. It acts as the orchestrator for the Entity Component System (ECS) worlds, managing their lifecycle, serialization, and hierarchical relationships between entities.

21.1 Scene Manager

The `SceneManager` class is the central authority for controlling which ECS world is currently active and how the engine transitions between different game states. It utilizes a stack-based approach for managing multiple active worlds, allowing for scenarios such as pausing a game to display a menu overlay.

21.1.1 World Stack Management

Internally, the `SceneManager` maintains a `m_worldStack`, which is a vector of unique pointers to `ECS::World` instances. This stack architecture enables complex state management:

```
std::vector<std::unique_ptr<ECS::World>> m_worldStack;
```

The primary methods for stack manipulation are:

- `switchScene(path, transition)`: Clears the current stack and loads a new scene from the specified path. This is the standard method for moving between levels.
- `pushScene(path)`: Loads a new scene and pushes it onto the top of the stack. The previous scene remains in memory but becomes inactive.
- `popScene()`: Removes the top-most scene from the stack, returning control to the scene immediately below it.

21.1.2 Scene Transitions

To ensure smooth visual flow between scenes, the `SceneManager` includes a transition system. Transitions are configured using the `TransitionConfig` structure, which defines the visual style and timing of the switch.

```
struct TransitionConfig {
    TransitionType type      = TransitionType::Fade;
    f32            duration  = 0.5f;
    Color          fadeColor = {};
};
```

The engine supports several `TransitionType` modes:

- `None`: Instantaneous swap.
- `Fade`: Smooth color overlay (usually black) that fades out the old scene and fades in the new one.
- `Slide`: Linear interpolation of viewports.
- `Custom`: Programmable transition effects.

During a transition, the `m_transitioning` flag is set to true, and `m_transProgress` is updated during the `update(dt)` loop based on the duration specified in the config.

21.1.3 Asynchronous Preloading

For large-scale levels, the engine supports preloading scenes into memory before they are needed. The `loadScene(path, async)` method can be called to populate the `m_preloaded` map.

```
SceneHandle loadScene(const char* path, bool async = false);
```

This returns a `SceneHandle`, which can later be used to activate the scene instantly, bypassing the disk I/O bottleneck during critical gameplay moments.

Design Rationale: Stack vs Single Scene While many engines only support one active scene, Caffeine uses a stack (`m_worldStack`) to facilitate UI and Sub-state management. By pushing a "PauseMenu" world onto the stack, the "GameWorld" state is preserved exactly as it was, avoiding expensive save/load cycles for simple overlays.

21.2 Scene Serialization

The `SceneSerializer` is responsible for converting the dynamic state of an `ECS::World` into a persistent format and vice versa. It employs a dual-format strategy to balance production performance with development flexibility.

21.2.1 Dual Format Strategy

Caffeine utilizes two distinct serialization formats:

1. **Binary (.caf)**: The primary format for production builds. It is designed for maximum loading speed and minimal file size. It uses raw memory copies for POD (Plain Old Data) components and a custom block-based structure.
2. **JSON**: A human-readable format used during development. It allows developers to inspect scene files in text editors, track changes via version control systems like Git, and manually tweak component values without needing a dedicated editor.

21.2.2 The .caf Binary Format

The binary format is structured around the `CafHeader` and a series of data blocks produced by `buildBlocks()`. The file layout consists of:

1. **Header**: Contains the magic number (`0xCAF0`), versioning information, and the asset type identifier.
2. **Metadata Block**: Stores high-level scene information such as entity counts and archetype offsets.
3. **Payload Block**: The raw component data, organized into typed sections.

The payload is reconstructed by the `parsePayload()` function. Each section in the payload starts with a `typeId` and a `count`, followed by the entity ID and component data for each entry.

```
// Example of binary section layout
struct RawSection {
    u32      typeId;
    u32      count;
    // Followed by count * (u32 eid + componentData)
};
```

21.2.3 Serialization Process

When `serialize()` is invoked, the engine performs the following steps:

1. **Block Building**: `buildBlocks()` iterates through all component types in the world (Transform, Health, etc.) and collects them into contiguous buffers.
2. **CRC Calculation**: A CRC32 checksum is generated for the payload to ensure data integrity.
3. **I/O**: The `CafWriter` handles the final writing to disk, ensuring proper alignment and header construction.

21.3 Scene Components and Hierarchy

Beyond simple data storage, the scene system implements a spatial hierarchy through specialized components found in `SceneComponents.hpp`.

21.3.1 The Parent Component

The `Parent` component establishes a relationship between two entities. It contains a handle to the parent entity and a dirty flag used for transform updates.

```
struct Parent {
    ECS::Entity parent = ECS::Entity::INVALID;
    bool        dirty  = true;
};
```

When an entity has a `Parent` component, its local `Transform` is treated as an offset relative to the parent, rather than a world-space position.

21.3.2 WorldTransform and Dirty Propagation

The `WorldTransform` component stores the final, calculated 4x4 matrix used for rendering.

```
struct WorldTransform {
    Mat4 matrix = Mat4::identity();
};
```

Dirty Flag Propagation: The system uses a "push" model for transform updates. When a parent's `Transform` changes, its `Parent::dirty` flag (and the flags of all its children) must be set to true.

During the scene update phase:

1. The system identifies all entities with `dirty == true`.
2. It traverses the hierarchy from the root down.
3. For each entity, it computes: $WorldMatrix = ParentWorldMatrix \times LocalTransformMatrix$
4. The result is stored in the `WorldTransform` component, and the dirty flag is cleared.

This approach ensures that matrix multiplications are only performed when necessary, significantly optimizing performance in complex scenes with deep hierarchies.

Implementation Detail: Entity Remapping During deserialization, entity handles in the file may not match the handles assigned by the current `ECS::World`. The `SceneSerializer::parsePayload` function maintains an `unordered_map<u32, ECS::Entity>` to remap old IDs to new ones, ensuring that the `Parent` components correctly point to the newly created entities in the current session.

Chapter 22

Animation System

The Caffeine Engine's Animation subsystem is a data-oriented, state-driven framework designed to handle 2D sprite-based animations with support for state transitions, blending, and frame-level events. By separating animation data (clips) from state logic (transitions) and playback state (animator), the system achieves a flexible and performant architecture that integrates seamlessly with the Entity-Component-System (ECS) through the `AnimationSystem`.

22.1 Foundational Structures

The system is built upon several key structures that define how animation data is stored and interpreted. At the lowest level, individual frames are represented by rectangle definitions.

22.1.1 FrameRect and AnimationClip

The `FrameRect` struct defines a source rectangle within a texture atlas. This allows the renderer to extract the correct sub-image for a specific frame of animation.

```
struct FrameRect {
    f32 x = 0.0f;
    f32 y = 0.0f;
    f32 w = 0.0f;
    f32 h = 0.0f;
};
```

An `AnimationClip` is a sequence of `FrameRect` objects accompanied by playback metadata. It serves as the raw asset from which animations are played.

```
struct AnimationClip {
    FixedString<32>         name;
    u32                     fps    = 12;
```

```
    std::vector<FrameRect> frames;
    bool                   loop   = true;

    f32 duration() const;
};
```

- **fps**: Frames per second, determining the playback speed of the clip.
- **frames**: A vector containing the source rectangles for each frame.
- **loop**: A boolean indicating whether the animation should restart once it reaches the end.

The duration of a clip is calculated based on its frame count and playback frequency:

$$\text{duration} = \frac{\text{frames.size()}}{\text{fps}} \tag{22.1}$$

22.1.2 Animation States and Transitions

Higher-level logic is handled by `AnimationState` and `AnimationTransition`. These structures define how the engine moves between different clips (e.g., from an "Idle" state to a "Run" state).

```
struct AnimationTransition {
    FixedString<32>        toState;
    std::function<bool()> condition;
    f32                    blendTime    = 0.1f;
    bool                   hasExitTime = false;
};
```

A transition defines a destination state (`toState`) and a predicate (`condition`) that must be met for the transition to trigger. The `hasExitTime` property is crucial for animations that must complete their current loop before transitioning (such as a jump start or an attack animation).

Design Rationale: Transition Evaluation The use of `std::function<bool()>` for transition conditions allows for complex logic to be defined by gameplay scripts or AI controllers. While it introduces a small overhead compared to pure data-driven flags, it provides the flexibility required for dynamic state machines where conditions might involve multiple external components.

The `AnimationState` wraps a clip and manages its specific playback parameters and outgoing transitions.

```
struct AnimationState {
    FixedString<32>                    name;
    const AnimationClip*               clip   = nullptr;
    f32                                speed  = 1.0f;
    std::vector<AnimationTransition>   transitions;
};
```

22.2 The Animator Component

The `Animator` is the primary ECS component used to attach animation capabilities to an entity. It maintains the current state machine configuration and tracks the progression of time.

```
struct Animator {
    HashMap<FixedString<32>, AnimationState>     states;
    FixedString<32>                              currentState;
    FixedString<32>                              previousState;
    f32                                          timeInState   =
       0.0f;
    f32                                          blendWeight   =
       1.0f;
    f32                                          playbackScale =
       1.0f;
    bool                                         paused        =
       false;
    std::vector<std::pair<u32, FixedString<32>>> frameEvents;
    std::function<void(const FixedString<32>&)>  onFrameEvent;
};
```

- **states**: A hash map storing the available states for this animator, indexed by their name.
- **timeInState**: Accumulates the elapsed time since the last state change, used to calculate the current frame.
- **playbackScale**: A global multiplier applied to the playback speed of any active state.
- **frameEvents**: A list of specific frames that trigger a callback. This is useful for synchronization (e.g., triggering a footstep sound on frame 3).

22.3 Animation System Logic

The `AnimationSystem` is responsible for updating all `Animator` components in the `World`. It operates on entities that possess both an `Animator` and a `Sprite` component.

22.3.1 State Machine Evaluation

During the update loop, the system first evaluates the transitions of the current state via `evaluateTransitions()`.

```
static void evaluateTransitions(Animator& anim) {
    const AnimationState* state =
       anim.states.get(anim.currentState);
    if (!state) return;

    for (const auto& t : state->transitions) {
        if (!t.condition) continue;
        if (t.hasExitTime && state->clip) {
            if (anim.timeInState < state->clip->duration())
               continue;
        }
        if (t.condition()) {
            anim.previousState = anim.currentState;
            anim.currentState  = t.toState;
            anim.timeInState   = 0.0f;
            return;
        }
    }
}
```

The evaluation logic respects the `hasExitTime` flag by checking if `timeInState` has reached the clip's duration. If a transition is triggered, the `timeInState` is reset to zero, effectively starting the new animation from its first frame.

22.3.2 Frame Index Computation

Once the current state is determined, the system calculates the appropriate frame index for the renderer. This involves applying state-specific and global speed multipliers to the delta time.

The effective playback speed is calculated as:

$$v_{\text{eff}} = \text{state.speed} \times \text{anim.playbackScale} \tag{22.2}$$

The time within the state is updated:

$$T_{\text{state}} = T_{\text{state}} + \Delta t \times v_{\text{eff}} \tag{22.3}$$

For looping animations, the time is wrapped within the clip's duration:

$$T_{\text{state}} = T_{\text{state}} \pmod{D_{\text{clip}}} \tag{22.4}$$

The final frame index F is then derived from the accumulated time and the clip's frame rate:

$$F = \lfloor T_{\text{state}} \times \text{fps} \rfloor \tag{22.5}$$

In non-looping mode, the frame index is clamped to ensure it does not exceed the bounds of the frame vector:

$$F_{\text{clamped}} = \min(F, \text{frameCount} - 1) \tag{22.6}$$

This logic ensures that animations remain smooth and synchronized with the engine's update frequency, even under varying frame rates.

22.3.3 Event Dispatching

The system checks for frame events by comparing the current frame index against the registered events in the `Animator`. If a match is found, the `onFrameEvent` callback is executed with the corresponding event name. This mechanism allows the engine to decouple visual feedback from gameplay logic while maintaining tight synchronization.

22.4 State Machine Workflow

A typical workflow for setting up an animation involves defining the clips, organizing them into a state machine, and configuring transitions.

Example: Player State Machine Consider a player character with "Idle", "Walk", and "Jump" states.

1. **Idle**: Loops indefinitely. Transitions to **Walk** when `velocity.x > 0`.
2. **Walk**: Loops indefinitely. Transitions to **Idle** when `velocity.x == 0`, and to **Jump** when `isGrounded == false`.
3. **Jump**: Does not loop (`loop = false`). Transitions back to **Idle** or **Walk** only after `hasExitTime` is met and the landing animation completes.

The blending window defined by `blendTime` in transitions allows the system to interpolate between the poses of the previous and current states, though the current 2D implementation primarily uses this for timing state swaps rather than skeletal bone blending.

22.5 Conclusion

The Caffeine Animation System provides a robust foundation for 2D character animation. By leveraging a state-machine architecture and formalizing the relationship between time, frame rates, and transitions, it allows developers to create complex, responsive animations with minimal boilerplate. The integration with the ECS ensures that animation logic is processed efficiently in parallel with other game systems, maintaining the engine's high-performance standards.

Chapter 23

Scripting System

The Caffeine Engine implements a specialized scripting subsystem designed for high performance and direct integration with the core C++ codebase. Unlike many modern engines that rely on external interpreted languages, Caffeine uses a native C++ scripting approach. This design choice ensures that gameplay logic runs at near metal speeds while maintaining full access to the engine's low level APIs and type safety.

23.1 Design Rationale

The decision to use C++ as the primary scripting language is central to the philosophy of the Caffeine Engine. By avoiding a virtual machine or a bytecode interpreter, the engine eliminates the traditional overhead associated with language bridging and data marshaling.

Performance and Type Safety Direct C++ scripting allows the compiler to optimize gameplay code alongside the engine core. Developers benefit from compile time checks, preventing a large class of runtime errors that are common in dynamically typed scripting environments. Furthermore, the absence of a garbage collector in the scripting layer provides predictable frame times, which is essential for maintaining a consistent 60 or 144 Hz simulation.

The scripting system is built around the concept of Hot Reloadable C++ modules. This provides the fast iteration cycles typically associated with interpreted languages while retaining the performance characteristics of compiled code.

23.2 The Script Engine

The `ScriptEngine` class serves as the central authority for managing the lifecycle of scripts within the engine. It handles the loading, unloading, and reloading of script modules, as well as the dispatching of events to active script instances.

23.2.1 Initialization and Configuration

To initialize the script engine, the `InitParams` structure must be populated with pointers to the core engine systems. This ensures that scripts have access to the world, input, and event subsystems from the moment they are created.

```
struct InitParams {
    ECS::World* world = nullptr;
    Input::InputManager* input = nullptr;
    Events::EventBus* events = nullptr;
};
```

Listing 23.1: ScriptEngine Initialization Parameters

23.2.2 The Scripting API

The `ScriptEngine` provides a streamlined API for interacting with the scripting layer. Most operations revolve around loading and managing script assets throughout the execution of the game.

- `loadScript(path, outError)`: Loads a C++ script module from the specified filesystem path. If the operation fails, it populates the optional error string with diagnostic information.
- `reloadScript(path, outError)`: Forces a reload of a previously loaded script. This method is critical for the hot reload workflow, as it handles the replacement of active script instances.
- `isLoaded(path)`: Returns a boolean indicating whether a script is currently registered and available for use.

These methods abstract the complexity of the underlying module loader, providing a simple interface for other engine systems to use.

23.2.3 ABI Stability via Pimpl Pattern

The `ScriptEngine` employs the Pointer to Implementation (Pimpl) pattern to maintain a stable Application Binary Interface (ABI). This technique encapsulates the internal state and third party dependencies within a private structure, ensuring that changes to the implementation do not require a recompilation of the entire engine.

```
class ScriptEngine {
public:
    // ... public API ...
private:
    struct Impl;
    std::unique_ptr<Impl> m_impl;
};
```

Listing 23.2: ScriptEngine Pimpl Implementation

This approach is particularly valuable for a scripting system where the underlying loading mechanisms or internal registries might evolve frequently.

23.3 The CppScript Base Class

Every user defined script in Caffeine must inherit from the `CppScript` base class. This class defines a set of virtual hooks that the engine calls at specific points in an entity's lifecycle.

```
class CppScript {
public:
    virtual ~CppScript() = default;

    virtual void onCreate(ECS::Entity entity, ECS::World& world);
    virtual void onUpdate(ECS::Entity entity, ECS::World& world,
       f32 dt);
    virtual void onDestroy(ECS::Entity entity, ECS::World& world);
    virtual void onCollision(ECS::Entity entity, ECS::Entity
       other, ECS::World& world);
};
```

Listing 23.3: CppScript Interface

By overriding these methods, developers can implement complex behaviors that react to world events, user input, or physics interactions.

23.3.1 Script Registration

To enable the engine to instantiate scripts by name, a registration macro is provided. This macro handles the boilerplate of adding the script factory to the global `CppScriptRegistry`.

```
class PlayerController : public CppScript {
    void onUpdate(ECS::Entity entity, ECS::World& world, f32 dt)
        override {
        // Gameplay logic here
    }
};

REGISTER_CPP_SCRIPT(PlayerController)
```

Listing 23.4: Registering a Custom Script

The registration happens at static initialization time, allowing the engine to discover all available scripts without manual configuration.

23.4 Script Components and Data

The integration of scripts into the Entity Component System (ECS) is handled via specialized components. These components store the necessary state to associate an entity with its scripted behavior.

23.4.1 ScriptComponent

The `ScriptComponent` is a lightweight structure that identifies the script associated with an entity. It primarily holds a path to the script file, which is used by the hot reload system to track changes.

```
struct ScriptComponent {
    std::string scriptPath;
};
```

Listing 23.5: ScriptComponent Structure

23.4.2 CppScriptComponent

For native C++ scripts, the `CppScriptComponent` maintains the actual instance of the script and its initialization state.

```
struct CppScriptComponent {
    std::string className;
    std::shared_ptr<CppScript> instance;
    bool initialized = false;
};
```

Listing 23.6: CppScriptComponent Structure

The `initialized` flag ensures that the `onCreate` hook is called exactly once, even if the script is added to an entity mid frame.

23.5 Systems and Runtime Logic

The execution of script logic is governed by the `ScriptSystem`, which is a standard ECS system that runs during the engine's update loop.

23.5.1 The ScriptSystem Implementation

The `ScriptSystem` bridges the gap between the `ScriptEngine` and the ECS world. It ensures that every entity with a script component receives the appropriate lifecycle callbacks at the right time.

```
void ScriptSystem::onUpdate(ECS::World& world, f32 dt) {
    auto view = world.view<CppScriptComponent>();
    for (auto entity : view) {
        auto& script = view.get<CppScriptComponent>(entity);

        if (!script.initialized) {
            m_engine->callOnCreate(script.className, entity);
            script.initialized = true;
        }

        m_engine->callOnUpdate(script.className, entity, dt);
    }
}
```

Listing 23.7: ScriptSystem Update Logic

This logic ensures that the `onCreate` hook is always called before the first `onUpdate`, providing a reliable initialization point for script state.

23.5.2 Interaction with Other Subsystems

The scripting system does not exist in isolation. Through the `InitParams` provided at startup, scripts can emit events via the `EventBus` or query the `InputManager` for user actions. This cross system communication allows scripts to serve as the glue that binds the engine's various technical modules into a cohesive game experience.

23.6 Native Callbacks and Performance

While `CppScript` classes provide the most structured way to write gameplay logic, the engine also supports `NativeScriptComponent` for scenarios requiring even tighter integration or lightweight closures.

```
struct NativeScriptComponent {
    ScriptInitCallback onCreate;
    ScriptCallback onUpdate;
    ScriptDestroyCallback onDestroy;
    ScriptCollisionCallback onCollision;
    bool initialized = false;
};
```

Listing 23.8: NativeScriptComponent Structure

These callbacks can be bound to lambda functions or static methods, offering a flexible alternative for simple behaviors that do not warrant a full class definition.

23.7 ScriptWatcher and Hot Reloading

One of the most powerful features of the Caffeine scripting system is its ability to reload scripts at runtime without restarting the engine. This is handled by the `ScriptWatcher` class.

The `ScriptWatcher` monitors the filesystem for changes to script files. When a modification is detected, it triggers the `ScriptEngine` to reload the corresponding module. The engine then identifies all active `CppScriptComponent` instances using that script, recreates their instances, and preserves their state where possible.

```
void ScriptWatcher::poll() {
    for (auto& [path, lastTime] : m_mtimes) {
        auto currentTime = std::filesystem::last_write_time(path);
        if (currentTime > lastTime) {
            m_engine->reloadScript(path);
            lastTime = currentTime;
```

```
            }
        }
}
```

Listing 23.9: ScriptWatcher Polling

This mechanism significantly reduces the feedback loop for gameplay programmers, allowing for rapid experimentation and tuning of game mechanics.

Chapter 24

User Interface System

The User Interface (UI) subsystem in the Caffeine Engine provides a flexible, component based framework for creating and managing graphical overlays. It operates within the Entity Component System (ECS) architecture, treating every UI element as an entity with specific visual and functional components. This design allows for seamless integration with the engine's core systems while maintaining a high degree of performance through data oriented layout and input processing.

24.1 Component Architecture

The UI system is built on top of a set of core structures defined in the `UI` namespace. These structures define the identity, appearance, and layout of widgets.

24.1.1 UIWidgetType

The `UIWidgetType` enumeration acts as a discriminator that identifies the functional role of a widget. The engine uses this type to determine how to render the widget and how to handle specific user interactions.

```
enum class UIWidgetType : u8 {
    Canvas,
    Panel,
    Button,
    Label,
    ProgressBar,
    Checkbox,
    Slider
};
```

Each type serves a unique semantic purpose within the interface hierarchy:

- **Canvas**: The root element of any UI tree. It defines the reference resolution and screen space for its descendants.
- **Panel**: A container widget used for grouping other elements. It often serves as a background or a clipping area.
- **Button**: An interactive element that responds to clicks and hovers, typically used to trigger actions.
- **Label**: A non interactive widget used for displaying text information.
- **ProgressBar**: A visual indicator of a scalar value relative to a range, such as health or loading progress.
- **Checkbox**: A toggleable widget that represents a boolean state.
- **Slider**: An interactive control for selecting a value from a continuous range.

24.1.2 Visual Styling

Visual properties are encapsulated in the `UIStyle` structure. This separation ensures that the visual appearance remains independent of the layout logic.

```
struct UIStyle {
    UIColor backgroundColor = {0.1f, 0.1f, 0.1f, 0.9f};
    UIColor textColor       = {1.0f, 1.0f, 1.0f, 1.0f};
    UIColor borderColor     = {0.3f, 0.3f, 0.3f, 1.0f};
    f32     borderWidth     = 1.0f;
    f32     borderRadius    = 4.0f;
    f32     fontSize        = 16.0f;
    Vec2    textAlignment   = {0.5f, 0.5f};
};
```

The `backgroundColor`, `textColor`, and `borderColor` fields use a normalized RGBA format. The `borderWidth` and `borderRadius` control the edge rendering, allowing for rounded corners. Text properties like `fontSize` and `textAlignment` guide the font renderer when processing labels or button captions.

24.2 The Layout Engine

Caffeine uses a hybrid layout system based on anchors and offsets. This system, represented by the `RectTransform` component, allows widgets to respond dynamically to parent resizing while maintaining precise pixel control where needed.

24.2.1 RectTransform Logic

Layout calculations depend on four vectors: `anchorMin`, `anchorMax`, `offsetMin`, and `offsetMax`. Anchors are defined as normalized coordinates (from 0 to 1) relative to the parent's bounding box. Offsets are defined in absolute pixels relative to those anchors.

```
struct RectTransform {
    Vec2 anchorMin = {0.0f, 0.0f};
    Vec2 anchorMax = {0.0f, 0.0f};
    Vec2 offsetMin = {0.0f, 0.0f};
    Vec2 offsetMax = {0.0f, 0.0f};
};
```

24.2.2 Layout Math Derivation

The core algorithm for computing a widget's screen space rectangle follows a unified formula. Let P_{min} and P_{max} be the minimum and maximum corners of the parent rectangle, and $S_P = P_{max} - P_{min}$ be the parent's size. The computed corners C_{min} and C_{max} are derived as:

$$C_{min} = P_{min} + (A_{min} \circ S_P) + O_{min} \tag{24.1}$$

$$C_{max} = P_{min} + (A_{max} \circ S_P) + O_{max} \tag{24.2}$$

Where $\circ$ denotes the Hadamard product (component wise multiplication). This math unifies relative and absolute positioning:

- **Full Stretch**: By setting $A_{min} = (0, 0)$ and $A_{max} = (1, 1)$ with zero offsets, the widget perfectly matches its parent's size.
- **Fixed Size, Top Left**: Setting $A_{min} = A_{max} = (0, 0)$ makes the corners relative to the top left. The size becomes $O_{max} - O_{min}$.
- **Fixed Size, Centered**: Setting $A_{min} = A_{max} = (0.5, 0.5)$ anchors the widget to the center of the parent.

Design Rationale: Anchor/Offset Duality The choice of an anchor/offset system eliminates the need for separate "pixel perfect" and "responsive" layout modes. By treating absolute offsets as deltas from normalized anchors, the engine can handle complex UI resizing, such as sidebars that stay fixed in width while the main content area expands to fill the remaining space.

24.3 System Logic

The `UISystem` manages the lifecycle of all UI components. It performs three primary tasks every frame: updating data bindings, recalculating layouts, and processing user input.

24.3.1 Layout Traversal

The `layoutWidgets` function is responsible for filling the `computedRect` field of every `UIWidget`. Since UI elements exist in a hierarchy, children must wait for their parents to be computed.

```
void layoutWidgets(ECS::World& world) {
    std::unordered_map<u32, UIRect> computed;
    // ... initial Canvas setup ...
    for (int pass = 0; pass < 8; ++pass) {
        world.forEach<UIWidget>(q, [&](ECS::Entity e, UIWidget& w)
           {
            if (w.type == UIWidgetType::Canvas) return;
            auto it = computed.find(w.parentId);
            if (it == computed.end()) return;
            UIRect rect = computeRect(w.transform, it->second);
            w.computedRect = rect;
            computed[e.id()] = rect;
        });
    }
}
```

The system uses an iterative multi pass approach rather than a recursive tree traversal. In each pass, it attempts to compute widgets whose parents are already known. An 8 pass limit is enforced, supporting hierarchies up to 8 levels deep. This avoids complex tree reconstruction in the ECS and keeps the layout logic linear.

24.3.2 Input Processing and Hit Testing

Input handling involves mapping screen space coordinates (from mouse or touch) to specific widgets. The process uses a hit testing algorithm that respects widget visibility and stacking order.

```
ECS::Entity hitTest(ECS::World& world, Vec2 screenPos) {
    ECS::Entity result = ECS::Entity::INVALID;
    i32 bestOrder = INT_MIN;
    world.forEach<UIWidget>(q, [&](ECS::Entity e, UIWidget& w) {
```

```
        if (!w.visible || !w.interactable) return;
        if (w.computedRect.contains(screenPos)) {
            if (w.siblingOrder > bestOrder) {
                bestOrder = w.siblingOrder;
                result = e;
            }
        }
    });
    return result;
}
```

When a click occurs, the system identifies the topmost widget under the cursor. Events then propagate to that widget's `onClick` callback. The `processInput` function also manages the hover state, triggering `onHoverEnter` and `onHoverExit` as the mouse moves across the interface.

24.3.3 Data Binding Mechanism

To keep the UI in sync with game logic without manual updates, the engine implements a simple data binding system. Widgets can be linked to external data sources using lambdas.

```
void bindValue(ECS::Entity widget, std::function<f32(ECS::World&)>
    getter) {
    ValueBinding b;
    b.widgetId = widget.id();
    b.getter   = std::move(getter);
    m_bindings.push_back(std::move(b));
}
```

The `updateBindings` function iterates through all registered bindings once per frame. It executes the getter to retrieve the current value and applies it to the corresponding component, such as `UIProgressBar::currentValue` or `UISlider::currentValue`. If the value has changed, the `onValueChanged` callback is triggered, allowing for reactive UI updates.

Chapter 25

Asset Management

The Asset Management subsystem in the Caffeine Engine is designed around the principles of asynchronous resource loading, zero-copy memory access, and automated lifetime management through reference counting. By decoupling the acquisition of a resource from its physical residence in memory, the engine maintains high frame rates during heavy I/O operations and ensures that memory fragmentation is minimized through the use of dedicated linear allocators for each asset.

25.1 The Asset Handle

The primary interface for interacting with the Asset Manager is the `AssetHandle<T>`. This template class serves as a RAII-compliant reference-counted handle to a resource of type T.

25.1.1 Reference Counting Semantics

`AssetHandle<T>` manages the lifetime of an asset by communicating with the `AssetManager`. The reference count is incremented whenever a handle is copied and decremented when it is destroyed.

$$R_{current} = \sum_{i=1}^{n} H_i \tag{25.1}$$

Where $R_{current}$ is the total reference count of an asset and n is the number of active `AssetHandle` instances pointing to that specific resource ID. When $R_{current}$ drops to zero, the asset becomes a candidate for eviction during the next garbage collection cycle.

25.1.2 Implementation Details

The handle is designed to be lightweight, containing only a pointer to the manager and the unique 32-bit identifier for the asset.

```
template<typename T>
class AssetHandle {
public:
    AssetHandle();
    AssetHandle(AssetManager* mgr, u32 id);
    ~AssetHandle();

    AssetHandle(const AssetHandle& o);
    AssetHandle& operator=(const AssetHandle& o);

    AssetHandle(AssetHandle&& o) noexcept;
    AssetHandle& operator=(AssetHandle&& o) noexcept;

    bool      isValid() const;
    bool      isReady() const;
    const T* get()     const;

    explicit operator bool() const;
    u32 id() const;
};
```

Listing 25.1: AssetHandle interface

The move constructor and move assignment operator are specifically optimized to transfer ownership without triggering atomic increments or decrements in the `AssetManager`. After a move, the source handle is set to a null state, ensuring that the destructor does not decrement the count for the transferred resource.

Design Rationale: Handle-Based Access By returning a handle instead of a raw pointer, the engine can safely perform hot-reloading or garbage collection in the background. The user never holds a direct pointer to the underlying data for longer than a single frame's scope, allowing the manager to invalidate or move memory without causing dangling pointers in game logic.

25.2 The Asset Manager

The `AssetManager` is the central authority for resource lifecycle. It maintains an internal registry of assets, indexed by their path, and coordinates with the `JobSystem` for background loading.

25.2.1 Loading Paths

The manager provides two distinct paths for loading resources: synchronous and asynchronous.

- **Sync Path:** Invoked via `loadSync<T>()`. This method calls `loadInternal()` immediately, blocking the calling thread until the asset is fully loaded and resolved. This is typically used during initial engine startup or loading screens where immediate availability is required.
- **Async Path:** Invoked via `loadAsync<T>()`. This method registers the asset requirement and calls `scheduleLoad()`, which submits a task to the `JobSystem`. The calling thread receives a handle immediately, but `isReady()` will return false until the background task completes.

25.2.2 Zero-Copy Memory Model

One of the most performance-critical features of the Caffeine Engine is its zero-copy asset storage. Each `AssetEntry` contains a `std::unique`$_ptr < LinearAllocator > thatownsthememoryslabwheretherawfiledataisloaded.$

$$M_{total} = \sum_{j=1}^{m} S_j + \Omega \tag{25.2}$$

Where M_{total} is the total memory footprint, S_j is the size of the allocator slab for asset j, and Ω represents the fixed overhead of the manager's tracking structures.

When an asset is loaded, the engine maps its structures directly onto the memory-mapped buffer. For example, a Texture object does not contain a copy of the pixel data; instead, it contains a pointer that points directly into the LinearAllocator slab.

```
struct AssetEntry {
    std::string                         path;
    AssetType                           cafType;
    std::atomic<LoadStatus>             status;
    std::unique_ptr<LinearAllocator>    allocator;
```

```
    const void*                     payload;
    ResolvedData                    resolved;
    std::atomic<u32>                refCount;
    u64                             sizeBytes;
};
```

Listing 25.2: AssetEntry Structure

25.2.3 Garbage Collection and Cache Management

The collectGarbage() function is responsible for pruning the cache. It iterates through the asset registry and identifies entries where the refCount is zero. These entries are evicted, and their associated LinearAllocator slabs are freed, returning memory to the system.

The manager also tracks performance metrics through the CacheStats structure:

```
struct CacheStats {
    u64 totalCachedBytes;
    u64 maxCacheBytes;
    u32 textureCount;
    u32 audioCount;
    u32 pendingJobs;
    f32 cacheHitRate;
};
```

Listing 25.3: CacheStats definition

The hit rate is calculated as:

$$HitRate = \frac{C_{hits}}{C_{total}} \tag{25.3}$$

where C_{hits} is the number of times an already-loaded asset was requested via path lookup, and C_{total} is the total number of load requests.

25.3 Asset Runtime Types

Caffeine utilizes a specialized .caf format for its assets. At runtime, these are represented as thin views into the loaded data.

25.3.1 Asset Mapping with Type Traits

To support a generic template-based API, the engine uses the AssetTypeTrait<T> mechanism. This allows the AssetManager to determine the internal AssetType discriminator at compile time.

```
template<> struct AssetTypeTrait<Texture> {
    static constexpr AssetType cafType = AssetType::Texture;
};
```

Listing 25.4: AssetTypeTrait Specialization

25.3.2 Specific View Structures

The three primary asset types currently supported by the engine are Texture, AudioClip, and ShaderBlob.

- Texture: Contains dimensions, format, and a pointer to the pixel buffer.

```
struct Texture {
    u32        width;
    u32        height;
    u32        format;
    u32        mipLevels;
    const u8* pixels;
    u64        pixelDataSize;
};
```

- AudioClip: Holds PCM data views and metadata for audio playback.

```
struct AudioClip {
    u32        sampleRate;
    u16        channels;
    u16        bitsPerSample;
    u32        sampleCount;
    const u8* pcmData;
    u64        pcmDataSize;
};
```

- ShaderBlob: A view into compiled shader bytecode.

```
struct ShaderBlob {
    u32        stage;
    const u8* bytecode;
```

```
    u64        bytecodeSize;
};
```

Optimization: Alignment and Padding When resolving these views, the AssetManager ensures that pointers (pixels, pcmData, bytecode) are aligned to the requirements of the underlying hardware (e.g., SIMD alignment for audio or GPU alignment for textures), even though they reside within a shared linear buffer.

Bibliography

Bibliography

[1] SDL3 GPU Category, official documentation. https://wiki.libsdl.org/SDL3/CategoryGPU

[2] J. Jylänki, ‘‘A Thousand Ways to Pack the Bin, A Practical Approach to Two-Dimensional Rectangle Bin Packing,’’ 2010.

[3] G. Fiedler, ‘‘Fix Your Timestep!,’’ *Gaffer on Games*, 2004. https://gafferongames.com/post/fix_your_timestep/

[4] K. Shoemake, ‘‘Animating Rotation with Quaternion Curves,’’ *SIGGRAPH 1985 Proceedings*, ACM, pp. 245, 254, 1985.

[5] D. Chase and Y. Lev, ‘‘Dynamic Circular Work-Stealing Deque,’’ *SPAA 2005*, pp. 21, 28, 2005.

[6] J. Baumgarte, ‘‘Stabilization of Constraints and Integrals of Motion in Dynamical Systems,’’ *Computer Methods in Applied Mechanics and Engineering*, vol. 1, no. 1, pp. 1, 16, 1972.

[7] CRC-32 IEEE 802.3 Standard (Ethernet), CRC-32 Polynomial definition. Also referenced in *IETF RFC 3720*.

[8] Khronos Group, ‘‘OpenGL Specification,’’ column-major matrix convention. https://www.khronos.org/opengl/

[9] D. Eberly, *3D Game Engine Architecture*, Morgan Kaufmann, 2005.

[10] B. Mirtich, ‘‘Impulse-Based Dynamic Simulation of Rigid Body Systems,’’ Ph.D. thesis, University of California, Berkeley, 1996.

[11] M. Dickheiser (ed.), *Game Programming Gems 6*, Charles River Media, 2006.

[12] Dear ImGui, https://github.com/ocornut/imgui

[13] ISO/IEC 14882:2020, *Programming Languages, C++*, 2020.